

Introduction to the Finite Element Method

Theory, Implementation, and MATLAB Examples

Author Name

Department of Mechanical Engineering

May 4, 2026

Dedicated to all students learning structural mechanics.

Preface

This book provides a concise introduction to the Finite Element Method (FEM) for structural and mechanical engineering applications. It is intended for senior undergraduate and graduate students who have a background in solid mechanics and linear algebra.

Each chapter introduces new element types, derives the governing equations from first principles, and concludes with a worked example. MATLAB code listings in Chapter [A](#) demonstrate practical implementation of the key algorithms.

The topics covered are:

- **Chapter 2** — Weighted residual methods for an axially loaded bar
- **Chapter 3** — One-dimensional bar and rod elements
- **Chapter 4** — Two-dimensional truss elements
- **Chapter 5** — Euler-Bernoulli beam and frame elements
- **Chapter 6** — Two-dimensional continuum elements (CST and isoparametric quad)
- **Chapter 7** — Structural dynamics, mass matrices, and time integration

March 2026

Contents

Preface	iii
1 Introduction to the Finite Element Method	1
1.1 Purpose of the Finite Element Method	1
1.2 Historical Background	1
1.3 Basic Idea	1
1.4 General FEM Procedure	2
1.5 Role of Approximation	2
1.6 Organization of This Book	3
1.7 Notation Used in This Book	3
2 Weighted Residual Methods	5
2.1 Motivation	5
2.2 Axially Loaded Bar	5
2.2.1 Equilibrium of a Differential Element	6
2.2.2 Constitutive and Kinematic Relations	6
2.3 Collocation Method	7
2.4 Subdomain Method	8
2.5 Galerkin Method	8
2.6 Comparison with the Exact Solution	8
2.7 MATLAB Implementation and Results	9
2.8 Connection to FEM	11
3 One-Dimensional Bar and Rod Elements	13
3.1 Direct Formulation of the Bar Element	13
3.1.1 Element Kinematics	13
3.1.2 Stiffness Matrix from Equilibrium	14
3.1.3 Unit Displacement Interpretation	14
3.2 Displacement Field Approximation	14
3.2.1 Linear Shape Functions	15
3.2.2 Enrichment by a Particular Solution	15
3.2.3 Worked Example: Uniform Distributed Load	16
3.3 Element Load Vector	16
3.4 Global Assembly	16
3.5 Boundary Conditions	17
3.5.1 Homogeneous Essential BC (fixed node)	17
3.5.2 Penalty Method	17
3.6 Worked Example: Three-Element Bar	17
3.7 Worked Example: Stepped Bar with Combined Loading	18

3.8	Principle of Virtual Work for the Axial Bar	21
3.8.1	Statement of the Principle	21
3.8.2	Derivation from Equilibrium	21
3.8.3	Recovery of the Bar Stiffness Matrix	22
3.8.4	Worked Example: Verification of PVW on the Stepped Bar	22
3.9	Galerkin Finite Element Formulation of the Bar	24
3.9.1	From Weighted Residuals to Shape Functions	24
3.9.2	Galerkin Weak Form for a Single Element	24
3.9.3	Element Stiffness Matrix and Load Vector	25
3.10	Equivalence of Galerkin and Virtual Work Formulations	25
3.11	Principle of Minimum Potential Energy	26
3.11.1	Total Potential Energy of the Axial Bar	26
3.11.2	Stationarity Condition and the Weak Form	26
3.11.3	Element Stiffness Matrix and Load Vector from PMPE	27
3.11.4	Summary: Four Equivalent Paths to the Same Element Matrices	27
3.12	Natural Coordinates and Isoparametric Mapping	28
4	2D Truss Element	29
4.1	2D Truss Element	29
4.1.1	Local Stiffness Matrix	29
4.1.2	Coordinate Transformation	29
4.1.3	Global Element Stiffness	29
4.2	Worked Example: Three-Element Planar Truss	30
5	Euler-Bernoulli Beam and Frame Elements	33
5.1	Euler-Bernoulli Beam Element	33
5.1.1	Degrees of Freedom	33
5.1.2	Hermite Shape Functions	33
5.1.3	Beam Stiffness Matrix	33
5.1.4	Consistent Load Vector (Uniform Load q)	34
5.2	Frame Element	34
5.3	Worked Example: Simply Supported Beam	34
6	Two-Dimensional Continuum Elements	35
6.1	Plane Elasticity	35
6.1.1	Stress and Strain Vectors	35
6.1.2	Constitutive Matrix	35
6.2	Constant Strain Triangle (CST)	35
6.2.1	Shape Functions	36
6.2.2	Strain-Displacement Matrix	36
6.2.3	Element Stiffness Matrix	36
6.3	Isoparametric Quadrilateral Element (Q4)	37
6.3.1	Shape Functions in Natural Coordinates	37
6.3.2	Jacobian Mapping	37
6.3.3	Numerical Integration (Gauss Quadrature)	37
6.4	Worked Example: CST Plate in Tension	37

7	Structural Dynamics and Vibration	39
7.1	Equations of Motion	39
7.2	Mass Matrix	39
7.2.1	Consistent Mass Matrix	39
7.2.2	Lumped Mass Matrix	40
7.3	Free Vibration: Eigenvalue Problem	40
7.4	Modal Analysis	40
7.4.1	Modal Decomposition	40
7.4.2	Damping: Rayleigh Model	41
7.5	Time Integration: Newmark- β Method	41
7.5.1	Effective Stiffness Matrix	41
7.6	Connection to 2-DOF System	41
A	MATLAB Code Listings	43
A.1	1D Bar Element Assembly and Solution	43
A.2	Stepped Bar with Distributed and Concentrated Loads	44
A.3	Verification of the Principle of Virtual Work	46
A.4	2D Truss Analysis	48
A.5	2D Beam Element	50
A.6	CST Element for Plane Stress	51
A.7	Newmark- β Time Integration for 2-DOF System	52

List of Figures

2.1	Fixed-free bar subjected to a nonuniform axial load.	5
2.2	Free-body diagram of a differential bar element.	6
2.3	MATLAB comparison of collocation, subdomain, and Galerkin one-term approximations against the exact solution.	10
3.1	Two-node bar element: nodal displacements u_1, u_2 (blue, above) and external nodal forces F_1, F_2 (red, below), with cross-section A , modulus E , length L	13
3.2	Three-element bar discretization: fixed at node 1, tip load F at node 4.	18
3.3	Stepped bar problem: distributed axial load (blue) on element 1 and concentrated forces (red) at nodes 2 and 3.	18
3.4	Graphical assembly of the global stiffness matrix. Blue cells receive contributions from element 1, red cells from element 2, and the violet cell at row/column 2 receives contributions from both, giving $94,500 + 236,250 = 330,750$	19
3.5	Displacement field $u(x)$ along the stepped bar.	20
3.6	Internal axial force $N(x)$ along the stepped bar. The discontinuity at $x = 500$ mm corresponds to the +6 kN concentrated force applied at node 2.	20
4.1	2D truss element inclined at angle θ : global axes (x, y) and local axes $(\bar{x}, \bar{y})$	30
4.2	Three-node, three-element planar truss. Node coordinates in mm. Pin supports at nodes 1 and 3; 10 kN downward load at node 2.	30
6.1	CST element: 3 nodes, 2 DOFs per node (u_i, v_i) , element area A_e	36
7.1	Two-DOF mass-spring-damper system corresponding to Equation (7.1) with matrices $\mathbf{M}, \mathbf{C}, \mathbf{K}$	39

List of Tables

2.1	Comparison of one-term weighted residual approximations.	9
3.1	Four derivation paths for the bar element matrices.	28
4.1	Element properties for the three-element truss.	31

Chapter 1

Introduction to the Finite Element Method

1.1 Purpose of the Finite Element Method

Many engineering problems are governed by differential equations together with boundary conditions. For simple geometries and idealized loading, these equations may have closed-form analytical solutions. In practical design, however, the geometry, material behavior, loading, and supports are often too complex for an exact solution.

The Finite Element Method (FEM) is a systematic numerical method for replacing a continuous problem by a finite set of algebraic equations. Instead of solving for an unknown field at infinitely many points, FEM solves for a finite number of unknowns at selected points called nodes. The solution between nodes is then interpolated by simple functions.

FEM is widely used in structural, mechanical, civil, aerospace, biomedical, and multiphysics engineering because it combines physical modeling with a practical computational procedure.

1.2 Historical Background

The modern finite element method developed in the 1950s from matrix structural analysis and the need to solve complex aerospace structures. Turner, Clough, Martin, and Topp [7] published an important early paper in 1956 describing the stiffness approach for plane stress analysis. The term *finite element* was introduced by Clough in 1960.

Since then, FEM has grown into one of the central tools of computational engineering. It is now implemented in commercial and open-source software for linear and nonlinear statics, heat transfer, dynamics, fluid flow, and coupled field problems.

1.3 Basic Idea

The central idea of FEM is discretization. A domain is divided into small, simple parts called **elements**. Elements meet at **nodes**, where the primary unknowns are stored. Within each element, the unknown field is approximated using interpolation functions called **shape functions**.

For a structural displacement problem, the final algebraic system commonly has the form

$$\mathbf{K} \mathbf{u} = \mathbf{f} \quad (1.1)$$

where $\mathbf{K}$ is the global stiffness matrix, $\mathbf{u}$ is the vector of unknown nodal displacements, and $\mathbf{f}$ is the global load vector.

Although Equation (1.1) looks compact, it represents the combined effect of element behavior, material properties, geometry, loading, and boundary conditions. Much of FEM is the careful construction of this system.

1.4 General FEM Procedure

A typical finite element analysis follows these steps:

1. **Define the physical problem:** Identify the domain, material properties, loads, and boundary conditions.
2. **Choose the element type:** Select elements appropriate for the physics and expected behavior of the solution.
3. **Discretize the domain:** Divide the domain into a mesh of finite elements connected at nodes.
4. **Form element equations:** Derive the element stiffness matrix and element load vector.
5. **Assemble the global system:** Combine all element equations into the global matrix equation.
6. **Apply boundary conditions:** Enforce prescribed values and include natural loading terms.
7. **Solve the algebraic system:** Compute the unknown nodal quantities.
8. **Post-process the results:** Recover derived quantities such as strains, stresses, reactions, or internal forces.

1.5 Role of Approximation

FEM is an approximate method. The accuracy of the result depends on the element formulation, mesh density, interpolation order, numerical integration, and the quality of the physical model. A refined mesh or higher-order interpolation can improve accuracy, but it also increases computational cost.

Good finite element modeling therefore requires engineering judgment. The goal is not only to obtain numbers, but to obtain numbers that are meaningful for the problem being studied.

1.6 Organization of This Book

This book begins with weighted residual methods because they provide a clear mathematical bridge from differential equations to finite element equations. It then develops one-dimensional elements, truss and beam elements, two-dimensional continuum elements, and basic structural dynamics.

Each chapter emphasizes the connection between theory, element formulation, and implementation. MATLAB examples in the appendix show how the main algorithms can be programmed directly.

1.7 Notation Used in This Book

Throughout this book, we use the following conventions:

Symbol	Meaning	Example
<i>u</i>	Column vector (bold italic)	displacement vector
<i>K</i>	Matrix (bold italic)	stiffness matrix
$(\cdot)^T$	Transpose	<i>K</i> ^T
$(\cdot)^{-1}$	Inverse	<i>K</i> ⁻¹
d	Differential operator	dx, dV
δ	Virtual quantity	δu, δW

Chapter 2

Weighted Residual Methods

2.1 Motivation

The finite element method can be introduced from several points of view: direct stiffness, energy methods, virtual work, or weighted residual methods. The weighted residual viewpoint is useful because it shows how an approximate solution is obtained when the differential equation cannot be satisfied exactly at every point of the domain.

Consider a trial solution $\tilde{u}(x)$ that satisfies the essential boundary conditions but is not the exact solution. After substitution into the governing differential equation, a nonzero residual remains:

$$R(x) = \mathcal{L}\tilde{u}(x) - f(x) \quad (2.1)$$

where $\mathcal{L}$ is the differential operator and $f(x)$ is the loading term. Weighted residual methods determine the unknown coefficients in $\tilde{u}$ by requiring the residual to vanish in an averaged or weighted sense:

$$\int_{\Omega} w_i(x) R(x) d\Omega = 0, \quad i = 1, 2, \dots, n \quad (2.2)$$

where $w_i(x)$ are weighting functions. Different choices of w_i lead to different methods.

2.2 Axially Loaded Bar

Consider the prismatic member in Figure 2.1, fixed at $x = 0$ and free at $x = L$. Since the load acts only along the axis of the member, the structural model is a bar or rod rather than a bending beam. The bar is subjected to a distributed axial load

$$p(x) = p_0 \left(\frac{x}{L} \right)^2 \quad (2.3)$$

with constant E and A .

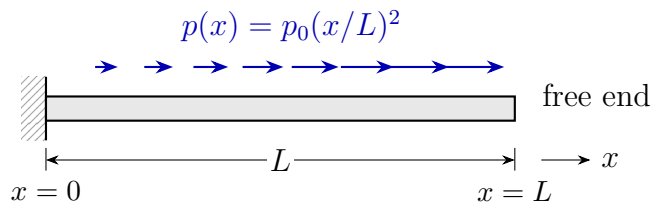


Figure 2.1: Fixed-free bar subjected to a nonuniform axial load.

2.2.1 Equilibrium of a Differential Element

To derive the governing equation, isolate a differential element of length dx as shown in Figure 2.2. Let $N(x)$ denote the internal axial force resultant. The sign convention is that $p(x)$ is positive in the positive x -direction and N is positive in tension.

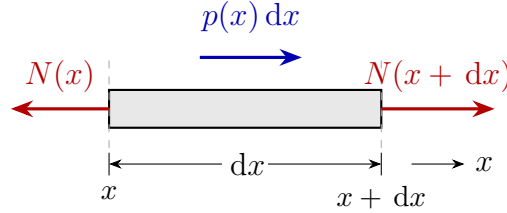


Figure 2.2: Free-body diagram of a differential bar element.

Force equilibrium in the axial direction gives

$$-N(x) + N(x + dx) + p(x) dx = 0. \quad (2.4)$$

Using the Taylor expansion

$$N(x + dx) = N(x) + \frac{dN}{dx} dx$$

and neglecting higher-order terms, Equation (2.4) becomes

$$\frac{dN}{dx} + p(x) = 0. \quad (2.5)$$

2.2.2 Constitutive and Kinematic Relations

The axial strain is related to the displacement field by

$$\varepsilon(x) = \frac{du}{dx}. \quad (2.6)$$

For a linear elastic material,

$$\sigma(x) = E\varepsilon(x). \quad (2.7)$$

The axial force resultant is

$$N(x) = \sigma(x)A = EA \frac{du}{dx}. \quad (2.8)$$

Substituting Equation (2.8) into Equation (2.5) gives the differential equation of an axially loaded bar:

$$\frac{d}{dx} \left(EA \frac{du}{dx} \right) + p(x) = 0. \quad (2.9)$$

Equivalently,

$$-\frac{d}{dx} \left(EA \frac{du}{dx} \right) = p(x). \quad (2.10)$$

For constant E and A , and for the load in Equation (2.3), the governing equation becomes

$$\begin{aligned} -EA \frac{d^2 u}{dx^2} &= p_0 \left(\frac{x}{L} \right)^2, & 0 < x < L \\ u(0) &= 0, & EAu'(L) = 0. \end{aligned} \quad (2.11)$$

The first boundary condition is essential because it prescribes displacement. The second is natural because it prescribes axial force. If an external axial force F were applied at $x = L$, the natural boundary condition would be $EAu'(L) = F$.

For this introductory comparison, use the one-term trial function

$$\tilde{u}(x) = a \left[\left(\frac{x}{L} \right)^2 - 2 \left(\frac{x}{L} \right) \right] \quad (2.12)$$

where a is the unknown coefficient. This function satisfies both boundary conditions:

$$\tilde{u}(0) = 0, \quad EA\tilde{u}'(L) = 0.$$

It is not flexible enough to satisfy the differential equation exactly, so a residual remains.

Let

$$\xi = \frac{x}{L}, \quad 0 \leq \xi \leq 1.$$

Substituting Equation (2.12) into Equation (2.11) gives

$$R(\xi) = -EA \frac{d^2 \tilde{u}}{dx^2} - p_0 \xi^2 = -\frac{2EA}{L^2} a - p_0 \xi^2. \quad (2.13)$$

The three methods below differ only in how they choose a from this residual.

2.3 Collocation Method

The collocation method requires the residual to vanish at selected points in the domain. Since the approximation contains one unknown coefficient, one collocation point is needed. Choosing the midpoint $\xi = 1/2$ gives

$$R\left(\frac{1}{2}\right) = 0. \quad (2.14)$$

Using Equation (2.13),

$$-\frac{2EA}{L^2} a - \frac{p_0}{4} = 0 \quad \Rightarrow \quad a_{\text{col}} = -\frac{p_0 L^2}{8EA}. \quad (2.15)$$

The approximate displacement is therefore

$$\tilde{u}_{\text{col}}(x) = -\frac{p_0 L^2}{8EA} (\xi^2 - 2\xi). \quad (2.16)$$

2.4 Subdomain Method

The subdomain method requires the residual to have zero average over each subdomain. With one unknown coefficient, the whole bar is taken as one subdomain:

$$\int_0^L R(x) dx = 0. \quad (2.17)$$

Equivalently,

$$L \int_0^1 \left(-\frac{2EA}{L^2}a - p_0\xi^2 \right) d\xi = 0. \quad (2.18)$$

Thus,

$$-\frac{2EA}{L^2}a - \frac{p_0}{3} = 0 \quad \Rightarrow \quad a_{\text{sub}} = -\frac{p_0L^2}{6EA}. \quad (2.19)$$

2.5 Galerkin Method

In the Galerkin method, the weighting function is chosen from the same family as the trial function. Here,

$$w(\xi) = \xi^2 - 2\xi. \quad (2.20)$$

The weighted residual statement is

$$\int_0^L w(x)R(x) dx = 0. \quad (2.21)$$

Substituting Equations (2.13) and (2.20) and changing to ξ gives

$$L \int_0^1 (\xi^2 - 2\xi) \left(-\frac{2EA}{L^2}a - p_0\xi^2 \right) d\xi = 0. \quad (2.22)$$

The required integrals are

$$\int_0^1 (\xi^2 - 2\xi) d\xi = -\frac{2}{3}, \quad \int_0^1 (\xi^2 - 2\xi)\xi^2 d\xi = -\frac{3}{10}.$$

Therefore,

$$\frac{4EA}{3L^2}a + \frac{3p_0}{10} = 0 \quad \Rightarrow \quad a_{\text{gal}} = -\frac{9p_0L^2}{40EA}. \quad (2.23)$$

Key Result

The Galerkin method is the weighted residual method that leads directly to the standard finite element formulation when piecewise polynomial trial functions are used and the governing equation is written in weak form.

2.6 Comparison with the Exact Solution

For this example, the exact solution is obtained by integrating Equation (2.11) twice and applying the boundary conditions:

$$u(x) = \frac{p_0L^2}{EA} \left(\frac{\xi}{3} - \frac{\xi^4}{12} \right). \quad (2.24)$$

Table 2.1: Comparison of one-term weighted residual approximations.

Method	Coefficient a	Tip displacement $\tilde{u}(L)$
Collocation	$-\frac{p_0 L^2}{8EA}$	$\frac{p_0 L^2}{8EA}$
Subdomain	$-\frac{6EA}{9p_0 L^2}$	$\frac{6EA}{9p_0 L^2}$
Galerkin	$-\frac{40EA}{9p_0 L^2}$	$\frac{40EA}{9p_0 L^2}$
Exact	—	$\frac{p_0 L^2}{4EA}$

At the free end,

$$u(L) = \frac{p_0 L^2}{4EA}. \quad (2.25)$$

Since $\tilde{u}(L) = -a$, the three approximations give the tip displacements listed in Table 2.1.

The collocation method makes the residual zero at one selected point, but the residual may still be large elsewhere. The subdomain method balances the total residual over the domain. The Galerkin method weights the residual with the same function used to approximate the displacement, which gives the best result among these one-term approximations for this problem.

2.7 MATLAB Implementation and Results

The following MATLAB script evaluates the coefficients, prints a numerical tip-displacement comparison, and generates the normalized displacement curves shown in Figure 2.3.

Figure 2.3 compares the three weighted residual approximations with the exact solution. The displacement is normalized as $uEA/(p_0 L^2)$, so the plot shows the method error independently of the chosen numerical values of E , A , p_0 , and L .

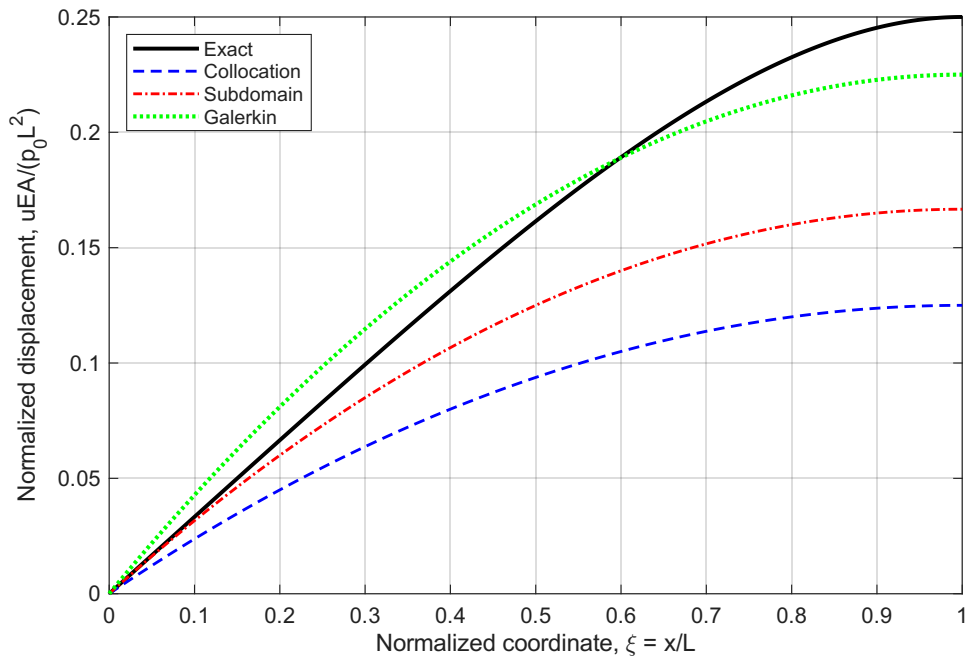


Figure 2.3: MATLAB comparison of collocation, subdomain, and Galerkin one-term approximations against the exact solution.

```

1 % weighted_residual_comparison.m
2 % One-term weighted residual approximation for an axially
   loaded bar
3
4 clear; clc; close all;
5
6 %% Problem parameters
7 L = 1.0;           % bar length [m]
8 E = 210e9;         % Young's modulus [Pa]
9 A = 1.0e-4;        % cross-sectional area [m^2]
10 p0 = 1.0e4;        % load intensity scale [N/m]
11 EA = E*A;          % axial stiffness [N]
12
13 %% Method coefficients for  $u_{\text{tilde}} = a*(xi^2 - 2*xi)$ 
14 a_col = -p0*L^2/(8*EA); % collocation at  $xi = 1/2$ 
15 a_sub = -p0*L^2/(6*EA); % one subdomain over  $[0,L]$ 
16 a_gal = -9*p0*L^2/(40*EA); % Galerkin weighting
17
18 methodNames = {'Collocation'; 'Subdomain'; 'Galerkin'};
19 aValues = [a_col; a_sub; a_gal];
20 tipValues = -aValues; %  $xi=1$  gives  $u_{\text{tilde}}(L) = -a$ 
21
22 %% Exact tip displacement and percent error
23 uTipExact = p0*L^2/(4*EA);
24 percentError = abs(tipValues - uTipExact)/uTipExact * 100;
25
26 fprintf('Weighted residual comparison for axial bar\n');
27 fprintf('L = %.3f m, EA = %.4e N, p0 = %.4e N/m\n\n', L, EA, p0
   );

```

```

28 fprintf('%-14s %14s %18s %12s\n', ...
29         'Method', 'a [m]', 'u_tilde(L) [m]', 'Error [%]');
30 fprintf('%s\n', repmat('-', 1, 62));
31
32 for i = 1:numel(methodNames)
33     fprintf('%-14s %14.6e %18.6e %12.3f\n', ...
34           methodNames{i}, aValues(i), tipValues(i), percentError(
35             i));
36 end
37 fprintf('%-14s %14s %18.6e %12.3f\n', 'Exact', '--', uTipExact,
38       0);
39
40 %% Normalized displacement curves
41 xi = linspace(0, 1, 400);
42 phi = xi.^2 - 2*xi;
43
44 u_col_norm = -(1/8) * phi;
45 u_sub_norm = -(1/6) * phi;
46 u_gal_norm = -(9/40) * phi;
47 u_exact_norm = xi/3 - xi.^4/12;
48
49 figure('Name','Weighted Residual Comparison','Color','w');
50 plot(xi, u_exact_norm, 'k-', 'LineWidth', 1.8); hold on;
51 plot(xi, u_col_norm, 'b--', 'LineWidth', 1.3);
52 plot(xi, u_sub_norm, 'r-.', 'LineWidth', 1.3);
53 plot(xi, u_gal_norm, 'g:', 'LineWidth', 1.9);
54 grid on; box on;
55 xlabel('Normalized coordinate, \xi = x/L');
56 ylabel('Normalized displacement, uEA/(p_0L^2)');
57 legend('Exact', 'Collocation', 'Subdomain', 'Galerkin', ...
58       'Location', 'northwest');

```

Listing 2.1: MATLAB comparison of weighted residual methods

2.8 Connection to FEM

The finite element method uses the Galerkin idea locally on each element. The unknown displacement field is approximated by shape functions,

$$u^e(x) = \mathbf{N}(x)\mathbf{u}^e, \quad (2.26)$$

and the weighting functions are chosen as the same shape functions. After integration by parts, the weak form for the axial bar becomes

$$\int_0^L EA \frac{dw}{dx} \frac{du}{dx} dx = \int_0^L wp dx + w(L)F. \quad (2.27)$$

This is the starting point for the bar element stiffness matrix derived in Chapter 3. In other words, the familiar finite element equations are not separate from weighted residual methods; they are the Galerkin weighted residual method applied element by element.

Chapter 3

One-Dimensional Bar and Rod Elements

3.1 Direct Formulation of the Bar Element

The simplest finite element is the two-node bar (rod) element, suitable for axially loaded members. Consider an element of length L , cross-sectional area A , and Young's modulus E , with nodes 1 and 2 having axial displacements u_1 and u_2 [5, 3].

Rather than starting from a variational principle, we derive the element stiffness directly from *mechanics of materials*: equilibrium of axial forces combined with Hooke's law. The principle of virtual work, derived later in this chapter (Section 3.8), reproduces the same stiffness matrix and provides the variational foundation for the entire finite element method.

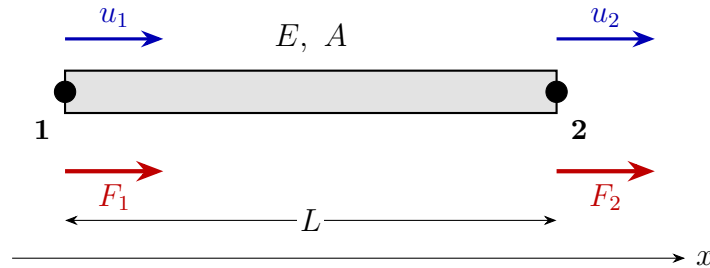


Figure 3.1: Two-node bar element: nodal displacements u_1 , u_2 (blue, above) and external nodal forces F_1 , F_2 (red, below), with cross-section A , modulus E , length L .

3.1.1 Element Kinematics

For small deformations, the axial elongation of the element is the difference of nodal displacements:

$$\Delta L = u_2 - u_1. \quad (3.1)$$

Assuming the strain is uniform along the element (consistent with linear displacement variation between nodes),

$$\varepsilon = \frac{\Delta L}{L} = \frac{u_2 - u_1}{L}. \quad (3.2)$$

Hooke's law and the axial force resultant $N = \sigma A$ give

$$\sigma = E\varepsilon = \frac{E(u_2 - u_1)}{L}, \quad N = \frac{EA(u_2 - u_1)}{L}. \quad (3.3)$$

3.1.2 Stiffness Matrix from Equilibrium

Taking $N > 0$ in tension, the external nodal forces required to produce the displacements u_1, u_2 are

$$F_1 = -N = \frac{EA}{L}(u_1 - u_2), \quad (3.4)$$

$$F_2 = +N = \frac{EA}{L}(u_2 - u_1). \quad (3.5)$$

Collecting these in matrix form,

$$\begin{bmatrix} F_1 \\ F_2 \end{bmatrix} = \frac{EA}{L} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} \iff \mathbf{k}^e \mathbf{u}^e = \mathbf{f}^e. \quad (3.6)$$

The element stiffness matrix is therefore

$$\mathbf{k}^e = \frac{EA}{L} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}. \quad (3.7)$$

Key Result

$$\mathbf{k}^e = \frac{EA}{L} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$$

3.1.3 Unit Displacement Interpretation

The columns of $\mathbf{k}^e$ have a clear physical meaning. Set $u_1 = 1$ and $u_2 = 0$: the bar shortens, the strain is $\varepsilon = -1/L$, and the internal force is $N = -EA/L$ (compression). Equilibrium then requires the external forces $F_1 = +EA/L$ at node 1 and $F_2 = -EA/L$ at node 2 — exactly the first column of $\mathbf{k}^e$. The second column follows from $u_1 = 0, u_2 = 1$ by an identical argument.

Remark 3.1. The stiffness matrix is symmetric (Maxwell's reciprocal theorem) and singular because rows sum to zero: a uniform translation $u_1 = u_2 = c$ produces no strain and no force, reflecting the rigid-body mode of the unconstrained element.

3.2 Displacement Field Approximation

The direct formulation has used only nodal displacements. To analyze the response *between* nodes — for postprocessing strains and stresses, or for representing distributed loading — we need an approximate displacement field $u(x)$.

3.2.1 Linear Shape Functions

The displacement field consistent with the constant-strain assumption used in Equation (3.2) is linear in x :

$$u(x) = N_1(x) u_1 + N_2(x) u_2 = \mathbf{N} \mathbf{u}^e \quad (3.8)$$

with the linear Lagrange shape functions

$$N_1(x) = 1 - \frac{x}{L}, \quad N_2(x) = \frac{x}{L}, \quad x \in [0, L]. \quad (3.9)$$

Remark 3.2. Shape functions satisfy the *partition of unity*, $N_1 + N_2 = 1$, and the *Kronecker delta property*, $N_i(x_j) = \delta_{ij}$.

The strain operator follows by differentiation,

$$\varepsilon(x) = \frac{du}{dx} = \mathbf{B} \mathbf{u}^e, \quad \mathbf{B} = \frac{1}{L} \begin{bmatrix} -1 & 1 \end{bmatrix}, \quad (3.10)$$

which recovers $\varepsilon = (u_2 - u_1)/L$, in agreement with Equation (3.2).

3.2.2 Enrichment by a Particular Solution

The strong form of the loaded bar, derived in Chapter 2, is

$$-EA \frac{d^2 u}{dx^2} = p(x), \quad 0 < x < L. \quad (3.11)$$

The linear interpolation Equation (3.8) solves the *homogeneous* equation $u'' = 0$ exactly, but cannot represent the inhomogeneous response when $p(x) \neq 0$.

To recover the exact response within the element, augment the linear field with a *particular solution* $u_p(x)$:

$$u(x) = N_1(x) u_1 + N_2(x) u_2 + u_p(x), \quad (3.12)$$

where $u_p(x)$ satisfies the inhomogeneous equation with homogeneous endpoint conditions:

$$-EA u_p''(x) = p(x), \quad u_p(0) = 0, \quad u_p(L) = 0. \quad (3.13)$$

The boundary conditions on u_p ensure that the linear part still recovers the nodal displacements at the endpoints, $u(0) = u_1$ and $u(L) = u_2$.

Key Result

Enriched trial function:

$$u(x) = N_1(x) u_1 + N_2(x) u_2 + u_p(x).$$

The first two terms carry the nodal displacements. The particular solution $u_p(x)$ carries the local response to the distributed load — solved once analytically per load case and added to every element under that loading.

3.2.3 Worked Example: Uniform Distributed Load

For a uniform distributed axial load $p(x) = p_0$, integrating Equation (3.13) twice and applying $u_p(0) = u_p(L) = 0$ gives

$$u_p(x) = \frac{p_0 x(L-x)}{2EA}. \quad (3.14)$$

This is a parabolic profile peaking at midspan with $u_p(L/2) = p_0 L^2/(8EA)$. Substituting into Equation (3.12) gives the complete element displacement field for a uniformly loaded bar:

$$u(x) = \left(1 - \frac{x}{L}\right) u_1 + \frac{x}{L} u_2 + \frac{p_0 x(L-x)}{2EA}. \quad (3.15)$$

Differentiating once gives the axial strain

$$\varepsilon(x) = \frac{u_2 - u_1}{L} + \frac{p_0(L-2x)}{2EA}, \quad (3.16)$$

and the internal axial force

$$N(x) = EA\varepsilon(x) = \frac{EA(u_2 - u_1)}{L} + \frac{p_0(L-2x)}{2}. \quad (3.17)$$

3.3 Element Load Vector

External forces at the element ends follow from equilibrium with the internal axial force in Equation (3.17): $F_1 = -N(0)$ and $F_2 = +N(L)$. Substituting,

$$F_1 = \frac{EA}{L}(u_1 - u_2) - \frac{p_0 L}{2}, \quad (3.18)$$

$$F_2 = \frac{EA}{L}(u_2 - u_1) - \frac{p_0 L}{2}. \quad (3.19)$$

Rearranging into the standard finite element form,

$$\mathbf{k}^e \mathbf{u}^e = \mathbf{f}^e, \quad \mathbf{f}^e = \frac{p_0 L}{2} \begin{bmatrix} 1 \\ 1 \end{bmatrix}. \quad (3.20)$$

Half of the total distributed load is placed at each node, recovering the familiar consistent load vector. Note that the same vector is obtained from the variational expression $\mathbf{f}^e = \int_0^L \mathbf{N}^T p \, dx$ used in Chapter 2 and in the PVW derivation of Section 3.8 — direct mechanics, weak form, and PVW all agree.

Remark 3.3. For constant EA and uniform p_0 , the enriched element with $u_p(x)$ is *nodally exact*: even a single element gives the exact analytical displacement and stress at the nodes. Refining the mesh only improves the resolution of the smooth interior fields, not the nodal accuracy.

3.4 Global Assembly

Consider a bar discretized into n elements with $n+1$ nodes. Each element connects local nodes 1–2 to global nodes i –($i+1$). The global stiffness matrix $\mathbf{K} \in \mathbb{R}^{(n+1) \times (n+1)}$ is assembled by the *direct stiffness method*:

$$\mathbf{K} = \sum_{e=1}^n \mathbf{L}_e^T \mathbf{k}^e \mathbf{L}_e \quad (3.21)$$

where $\mathbf{L}_e$ is the Boolean connectivity matrix that maps local to global DOFs.

In practice, assembly is performed by adding element stiffness entries at the appropriate global positions:

$$K_{ij} += k_{rs}^e, \quad \text{where } i = \text{global}(r), \quad j = \text{global}(s) \quad (3.22)$$

3.5 Boundary Conditions

3.5.1 Homogeneous Essential BC (fixed node)

Zero out the row and column corresponding to the prescribed DOF, and set the diagonal to 1:

$$K_{ii} = 1, \quad K_{ij} = K_{ji} = 0 \quad \forall j \neq i, \quad f_i = 0 \quad (3.23)$$

3.5.2 Penalty Method

Add a large number $\alpha \gg \max(K_{ii})$ to the diagonal:

$$K_{ii} += \alpha, \quad f_i += \alpha \bar{u}_i \quad (3.24)$$

where $\bar{u}_i$ is the prescribed displacement.

3.6 Worked Example: Three-Element Bar

Example 3.1 (Three-element bar under tip load). Consider a stepped bar of total length $L = 0.9$ m fixed at $x = 0$, with a point load $F = 10,000$ N at $x = L$. Three equal elements, $EA = 70 \times 10^6$ N.

Element stiffness:

$$\mathbf{k}^e = \frac{EA}{L/3} = \frac{70 \times 10^6}{0.3} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} = 2.3 \times 10^8 \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$$

Global system (4 DOFs, node 1 fixed):

$$\mathbf{K} = 2.3 \times 10^8 \begin{bmatrix} 1 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 1 \end{bmatrix}, \quad \mathbf{f} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 10000 \end{bmatrix}$$

Applying BC $u_1 = 0$ (strike row/column 1):

$$\mathbf{u} = \mathbf{K}^{-1} \mathbf{f} \implies u_4 = \frac{FL}{EA} = 1.286 \times 10^{-4} \text{ m}$$

See Section A.1 in the Appendix for the MATLAB implementation.

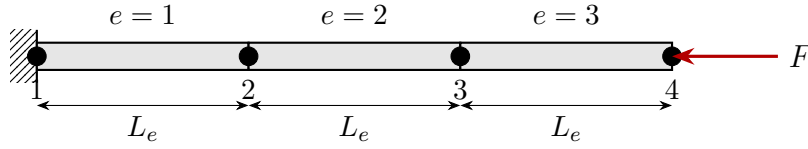


Figure 3.2: Three-element bar discretization: fixed at node 1, tip load F at node 4.

3.7 Worked Example: Stepped Bar with Combined Loading

Example 3.2 (Stepped axial bar with distributed and concentrated loads). A cantilever stepped bar comprises two prismatic segments. Element 1 has length $L_1 = 500$ mm and area $A = 225$ mm²; element 2 has length $L_2 = 400$ mm and area $2A = 450$ mm². The Young's modulus is $E = 210,000$ MPa. A uniform distributed axial load $p_1 = -4$ N/mm acts on element 1 (negative = directed toward the fixed support), and concentrated forces of $+6$ kN at node 2 and -15 kN at node 3 are applied (Figure 3.3).

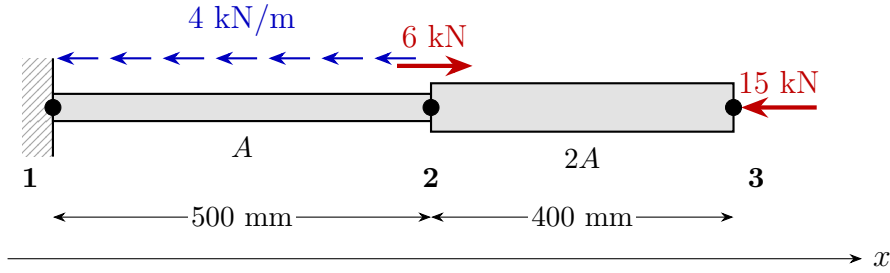


Figure 3.3: Stepped bar problem: distributed axial load (blue) on element 1 and concentrated forces (red) at nodes 2 and 3.

Discretization. Two linear bar elements connect three nodes located at $x_1 = 0$, $x_2 = 500$, $x_3 = 900$ mm. Each node carries one axial DOF, giving $n_{\text{dof}} = 3$.

Node coordinates		Connectivity		
Node	x [mm]	Element	Nodes	DOFs
1	0	1	1, 2	1, 2
2	500	2	2, 3	2, 3
3	900			

Element matrices. Using the direct stiffness from Equation (3.7),

$$\mathbf{k}^{(1)} = \frac{EA}{L_1} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} = 94,500 \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \frac{\text{N}}{\text{mm}}, \quad \mathbf{k}^{(2)} = \frac{E(2A)}{L_2} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} = 236,250 \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \frac{\text{N}}{\text{mm}}.$$

The consistent distributed-load vector for element 1, from Equation (3.20), is

$$\mathbf{f}_p^{(1)} = \frac{p_1 L_1}{2} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} -1000 \\ -1000 \end{bmatrix} \text{ N}, \quad \mathbf{f}_p^{(2)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \text{ N}.$$

Graphical assembly of the global stiffness matrix. Each 2×2 element matrix is added into the 3×3 global matrix at the rows and columns corresponding to its nodes (Figure 3.4): element 1 occupies the top-left block (rows/columns 1 and 2), element 2 the bottom-right block (rows/columns 2 and 3). The shared entry at row/column 2 receives contributions from both elements.

Element 1 contribution	Element 2 contribution	Global K																											
<table border="1"> <tr><td>94500</td><td>-94500</td><td>0</td></tr> <tr><td>-94500</td><td>94500</td><td>0</td></tr> <tr><td>0</td><td>0</td><td>0</td></tr> </table>	94500	-94500	0	-94500	94500	0	0	0	0	<table border="1"> <tr><td>0</td><td>0</td><td>0</td></tr> <tr><td>0</td><td>236250</td><td>-236250</td></tr> <tr><td>0</td><td>-236250</td><td>236250</td></tr> </table>	0	0	0	0	236250	-236250	0	-236250	236250	<table border="1"> <tr><td>94500</td><td>-94500</td><td>0</td></tr> <tr><td>-94500</td><td>330750</td><td>-236250</td></tr> <tr><td>0</td><td>-236250</td><td>236250</td></tr> </table>	94500	-94500	0	-94500	330750	-236250	0	-236250	236250
94500	-94500	0																											
-94500	94500	0																											
0	0	0																											
0	0	0																											
0	236250	-236250																											
0	-236250	236250																											
94500	-94500	0																											
-94500	330750	-236250																											
0	-236250	236250																											

Figure 3.4: Graphical assembly of the global stiffness matrix. Blue cells receive contributions from element 1, red cells from element 2, and the violet cell at row/column 2 receives contributions from both, giving $94,500 + 236,250 = 330,750$.

Force vector assembly. The distributed-load contribution from element 1 lands at DOFs 1 and 2; the concentrated forces are placed directly at their respective nodes:

$$\mathbf{f}_p = \begin{bmatrix} -1000 \\ -1000 \\ 0 \end{bmatrix}, \quad \mathbf{f}_c = \begin{bmatrix} 0 \\ 6000 \\ -15000 \end{bmatrix}, \quad \mathbf{f} = \mathbf{f}_p + \mathbf{f}_c = \begin{bmatrix} -1000 \\ 5000 \\ -15000 \end{bmatrix} \text{ N.}$$

Boundary conditions and reduced system. Node 1 is fixed, $u_1 = 0$. The free-DOF set is $\{2, 3\}$. Striking the first row and column,

$$\underbrace{\begin{bmatrix} 330,750 & -236,250 \\ -236,250 & 236,250 \end{bmatrix}}_{\mathbf{K}_{ff}} \underbrace{\begin{bmatrix} u_2 \\ u_3 \end{bmatrix}}_{\mathbf{f}_f} = \underbrace{\begin{bmatrix} 5,000 \\ -15,000 \end{bmatrix}}_{\mathbf{f}_f}.$$

Solving,

$$u_2 = -\frac{20}{189} \approx -0.1058 \text{ mm}, \quad u_3 = -\frac{32}{189} \approx -0.1693 \text{ mm}.$$

Reaction at the fixed support. The reaction follows from the first row of the unreduced system:

$$R_1 = \mathbf{K}_{1,*} \mathbf{u} - f_1 = (-94,500)\left(-\frac{20}{189}\right) - (-1000) = +11,000 \text{ N},$$

which balances the resultant of all applied axial forces: $-(-2000) - 6000 - (-15000) = +11,000 \text{ N}$.

Internal displacement and force fields. Inside each element the displacement follows the enriched trial Equation (3.12), evaluated with the nodal solution and the particular solution Equation (3.14). Differentiating yields $N(x)$ from Equation (3.17). The resulting curves (Figures 3.5 and 3.6) show the parabolic curvature in element 1 caused by $p_1 \neq 0$, the linear field in element 2, and the $+6 \text{ kN}$ jump in $N(x)$ at $x = 500 \text{ mm}$ corresponding to the concentrated load applied there.

See Section A.2 in the Appendix for the full MATLAB implementation that produced these plots.

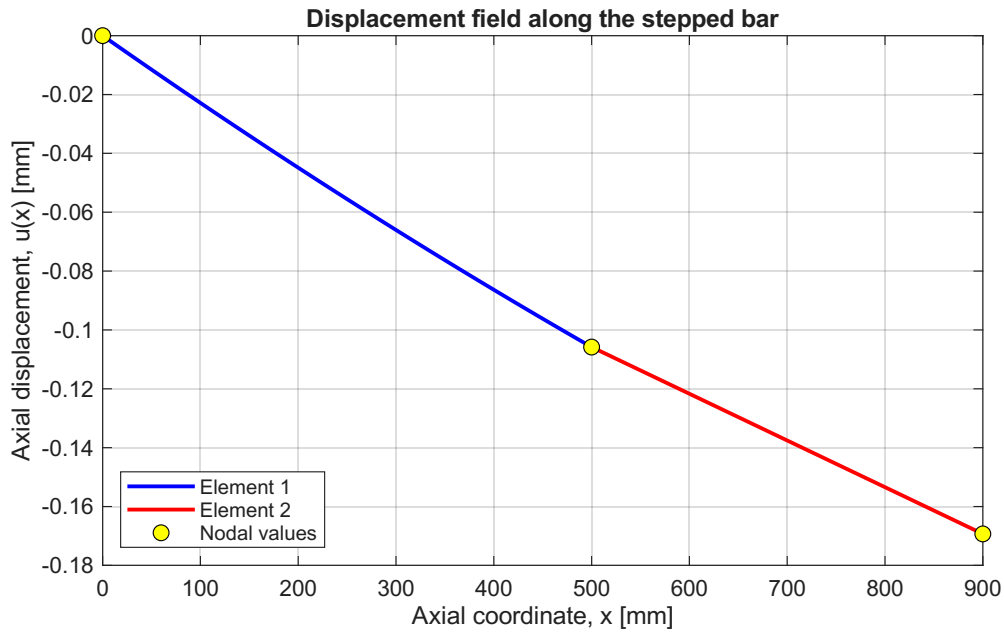


Figure 3.5: Displacement field $u(x)$ along the stepped bar.

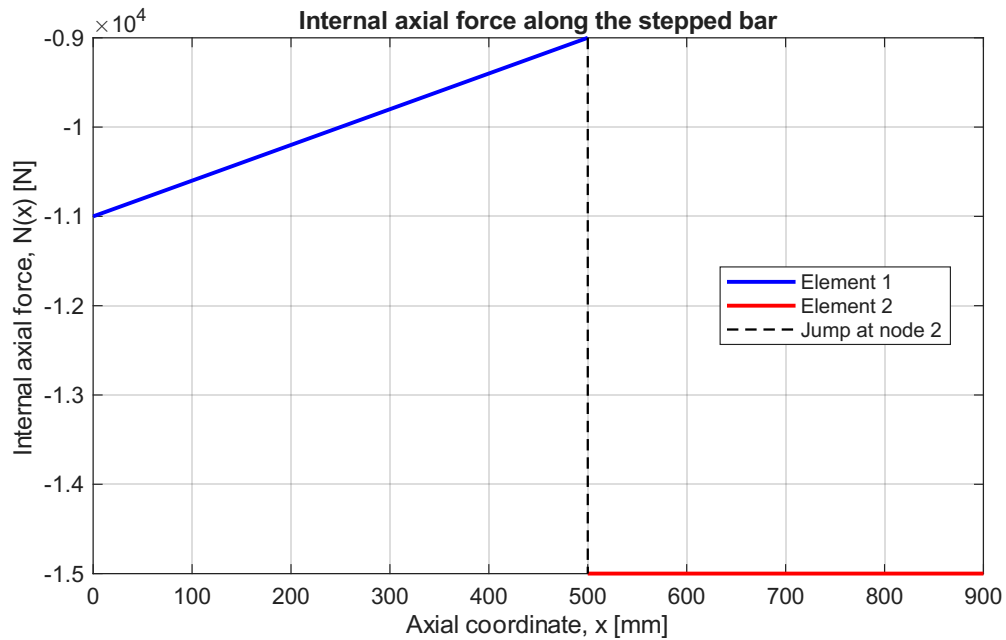


Figure 3.6: Internal axial force $N(x)$ along the stepped bar. The discontinuity at $x = 500$ mm corresponds to the +6 kN concentrated force applied at node 2.

3.8 Principle of Virtual Work for the Axial Bar

The direct formulation of Section 3.1 and the worked examples above gave the bar element stiffness and load vector from physical equilibrium. The principle of virtual work (PVW) provides an alternative, energy-based statement of equilibrium that reproduces the same results and generalizes immediately to non-uniform loading, variable cross-section, and higher-order elements. PVW is the variational foundation on which the entire finite element method rests.

3.8.1 Statement of the Principle

Key Result

Principle of Virtual Work: For a bar in static equilibrium, and for any *kinematically admissible* virtual displacement field $\delta u(x)$,

$$\delta W_{\text{int}} = \delta W_{\text{ext}}, \quad (3.25)$$

with internal and external virtual work

$$\delta W_{\text{int}} = \int_0^L N(x) \delta \varepsilon(x) \, dx = \int_0^L EA \frac{du}{dx} \frac{d\delta u}{dx} \, dx,$$

$$\delta W_{\text{ext}} = \int_0^L p(x) \delta u(x) \, dx + \sum_i F_i \delta u(x_i).$$

A virtual displacement is *kinematically admissible* if it satisfies the essential boundary conditions: $\delta u = 0$ wherever u is prescribed.

The virtual displacement $\delta u(x)$ is an arbitrary, infinitesimal perturbation. It need not satisfy equilibrium or any natural boundary conditions — its sole role is to test the equilibrium of the actual displacement field $u(x)$.

3.8.2 Derivation from Equilibrium

Begin with the local equilibrium equation derived in Chapter 2:

$$\frac{dN}{dx} + p(x) = 0, \quad 0 < x < L. \quad (3.26)$$

Multiply both sides by an arbitrary virtual displacement $\delta u(x)$ and integrate over the bar:

$$\int_0^L \left(\frac{dN}{dx} + p \right) \delta u \, dx = 0. \quad (3.27)$$

Integration by parts on the first term gives

$$\int_0^L \frac{dN}{dx} \delta u \, dx = \left[N(x) \delta u(x) \right]_0^L - \int_0^L N(x) \frac{d\delta u}{dx} \, dx. \quad (3.28)$$

Substituting back into Equation (3.27) and rearranging,

$$\int_0^L N \frac{d\delta u}{dx} \, dx = \left[N \delta u \right]_0^L + \int_0^L p \delta u \, dx. \quad (3.29)$$

The boundary term vanishes wherever the displacement is prescribed (kinematic admissibility forces $\delta u = 0$ there) and reduces to $F_i \delta u(x_i)$ at any free end with a concentrated force F_i (because N at that end equals the applied force). Substituting the constitutive law $N = EA \, du/dx$ and $\delta \varepsilon = d\delta u/dx$ on the left, equation Equation (3.29) becomes

$$\int_0^L EA \frac{du}{dx} \frac{d\delta u}{dx} dx = \int_0^L p \delta u dx + \sum_i F_i \delta u(x_i), \quad (3.30)$$

which is exactly the PVW statement Equation (3.25).

Remark 3.4. Equation Equation (3.30) is the *weak form* of the bar equilibrium equation. The strong form $-EA u'' = p$ requires twice-differentiable trial functions; the weak form requires only first derivatives. This relaxation makes the weak form well-suited to piecewise-polynomial finite element approximations whose derivatives may be discontinuous between elements.

3.8.3 Recovery of the Bar Stiffness Matrix

To reproduce the element stiffness derived in Section 3.1, substitute the linear interpolation

$$u(x) = N_1(x) u_1 + N_2(x) u_2, \quad \delta u(x) = N_1(x) \delta u_1 + N_2(x) \delta u_2, \quad (3.31)$$

with strain-displacement matrix $\mathbf{B} = \frac{1}{L}[-1, 1]$, into the left-hand side of Equation (3.30):

$$\delta \mathbf{u}^e{}^\top \left[\int_0^L EA \mathbf{B}^\top \mathbf{B} dx \right] \mathbf{u}^e = \delta \mathbf{u}^e{}^\top \mathbf{k}^e \mathbf{u}^e,$$

giving

$$\mathbf{k}^e = \int_0^L EA \mathbf{B}^\top \mathbf{B} dx = \frac{EA}{L} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix},$$

identical to Equation (3.7). The right-hand side likewise produces the consistent load vector $\mathbf{f}^e = \int_0^L \mathbf{N}^\top p dx + (\text{nodal forces})$. Direct mechanics, weak form, and PVW all give the same element equations.

3.8.4 Worked Example: Verification of PVW on the Stepped Bar

Example 3.3 (Numerical verification of $\delta W_{\text{int}} = \delta W_{\text{ext}}$). To illustrate that PVW holds for a real loaded bar in equilibrium, we return to the stepped bar of Example 3.2 (already solved exactly). Recall the actual nodal solution

$$u_1 = 0, \quad u_2 = -\frac{20}{189} \approx -0.10582 \text{ mm}, \quad u_3 = -\frac{32}{189} \approx -0.16931 \text{ mm},$$

the support reaction $R_1 = +11,000$ N, and the internal axial force per element from Equation (3.17),

$$\begin{aligned} N^{(1)}(x_1) &= \frac{EA_1(u_2 - u_1)}{L_1} + \frac{p_1(L_1 - 2x_1)}{2} = -10,000 + (-2)(500 - 2x_1) = -11,000 + 4x_1 \text{ [N]}, \\ N^{(2)}(x_2) &= \frac{EA_2(u_3 - u_2)}{L_2} = -15,000 \text{ [N]} \quad (\text{constant}), \end{aligned}$$

where $x_1 \in [0, 500]$ and $x_2 \in [0, 400]$ are local element coordinates.

We now choose two different kinematically admissible virtual displacements $\delta u(x)$ (each must satisfy $\delta u(0) = 0$) and verify the equality $\delta W_{\text{int}} = \delta W_{\text{ext}}$ by direct calculation of both sides.

Case A: $(\delta u_1, \delta u_2, \delta u_3) = (0, 1, 0)$ “**tent**”. Piecewise linear with the apex at node 2:

$$\delta u^{(1)}(x_1) = \frac{x_1}{L_1} = \frac{x_1}{500}, \quad \delta u^{(2)}(x_2) = 1 - \frac{x_2}{L_2} = 1 - \frac{x_2}{400}.$$

The corresponding virtual strains are constant on each element:

$$\delta \varepsilon^{(1)} = +\frac{1}{500}, \quad \delta \varepsilon^{(2)} = -\frac{1}{400}.$$

Internal virtual work. For an element with constant EA and uniform p , $\int_0^L N(x) \, dx = EA(u_b - u_a)$ (the distributed-load contribution integrates to zero). Hence

$$\delta W_{\text{int}}^{(1)} = \delta \varepsilon^{(1)} \int_0^{L_1} N^{(1)} \, dx_1 = \frac{1}{500} EA_1(u_2 - u_1) = \frac{1}{500} (-5,000,000) = -10,000 \text{ N mm},$$

$$\delta W_{\text{int}}^{(2)} = \delta \varepsilon^{(2)} \int_0^{L_2} N^{(2)} \, dx_2 = -\frac{1}{400} EA_2(u_3 - u_2) = -\frac{1}{400} (-6,000,000) = +15,000 \text{ N mm},$$

$$\delta W_{\text{int}} = -10,000 + 15,000 = +5,000 \text{ N mm}.$$

External virtual work. The reaction at node 1 does no virtual work because $\delta u_1 = 0$.

$$\begin{aligned} \int_0^{L_1} p_1 \delta u^{(1)} \, dx_1 &= \int_0^{500} (-4) \left(\frac{x_1}{500} \right) dx_1 = -1,000 \text{ N mm}, \\ F_2 \delta u_2 + F_3 \delta u_3 &= (6,000)(1) + (-15,000)(0) = +6,000 \text{ N mm}, \\ \delta W_{\text{ext}} &= -1,000 + 6,000 = +5,000 \text{ N mm}. \end{aligned}$$

Result: $\delta W_{\text{int}} = \delta W_{\text{ext}} = +5,000 \text{ N mm}$. ✓

Case B: $(\delta u_1, \delta u_2, \delta u_3) = (0, 1, 1)$ “**ramp**”. Linear in element 1, constant in element 2:

$$\delta u^{(1)} = \frac{x_1}{500}, \quad \delta u^{(2)} = 1, \quad \delta \varepsilon^{(1)} = \frac{1}{500}, \quad \delta \varepsilon^{(2)} = 0.$$

$$\delta W_{\text{int}} = \frac{1}{500} (-5,000,000) + 0 = -10,000 \text{ N mm},$$

$$\delta W_{\text{ext}} = \underbrace{-1,000}_{\text{distributed load}} + \underbrace{(6,000)(1) + (-15,000)(1)}_{\text{concentrated forces}} = -1,000 - 9,000 = -10,000 \text{ N mm}.$$

Result: $\delta W_{\text{int}} = \delta W_{\text{ext}} = -10,000 \text{ N mm}$. ✓

The two virtual displacements above coincide with the basis used in the FEM discretization (piecewise linear, one unit value at a free DOF, zero elsewhere). Choosing δu as the i -th basis function reproduces exactly the i -th row of the reduced FEM equations $\mathbf{K}_{ff} \mathbf{u}_f = \mathbf{f}_f$. Indeed, +5,000 and −10,000 are the assembled effective forces at DOF 2 and DOF 2+3 for these two perturbations.

Beyond piecewise linear. PVW holds for any kinematically admissible virtual displacement, not just those built from the FEM basis. The MATLAB script Section A.3 also evaluates a smooth quadratic $\delta u(x) = (x/900)^2$ and reports

$$\delta W_{\text{int}} = \delta W_{\text{ext}} \approx -13,353.91 \text{ N mm} \quad (|\Delta| < 10^{-11} \text{ N mm}),$$

confirming that the work balance is independent of the chosen test field.

Remark 3.5. This verification is more than a numerical check — it is the defining property of the equilibrium configuration. Of all kinematically admissible displacement fields, the actual $u(x)$ is the unique one for which the work balance holds for *every* admissible perturbation. The finite element method exploits this property: by enforcing PVW with the basis functions of the chosen mesh as test fields, it produces the discrete equations $\mathbf{K}\mathbf{u} = \mathbf{f}$ whose solution is the best approximation to the true equilibrium within that finite-dimensional space.

3.9 Galerkin Finite Element Formulation of the Bar

Remark 3.6. Chapter 2 applied the Galerkin method with a single global polynomial – one free parameter, one weight function – to the continuous bar. The formulation developed here uses the finite element shape functions N_1, N_2 of Equation (3.9) as both trial and test functions on each element. With piecewise-linear shape functions and one test function per node, this is the Galerkin principle applied at the element level: the method through which every finite element stiffness integral is derived in practice.

3.9.1 From Weighted Residuals to Shape Functions

Start from the strong form of the loaded bar (Equation (3.11)):

$$-EA \frac{d^2 u}{dx^2} = p(x), \quad 0 < x < L.$$

Substitute the finite element approximation $u^h(x) = \mathbf{N}(x) \mathbf{u}^e$ from Equation (3.8). Because N_1 and N_2 are linear in x , their second derivative vanishes identically, so the residual is

$$R(x) = -EA \frac{d^2 u^h}{dx^2} - p(x) = -p(x) \neq 0. \quad (3.32)$$

The linear approximation cannot satisfy the ODE pointwise when $p(x) \neq 0$; it satisfies it instead in an *average* sense. The **Galerkin statement** requires the residual to be orthogonal to each shape function:

$$\int_0^L N_i(x) R(x) dx = 0, \quad i = 1, 2. \quad (3.33)$$

This choice – test functions identical to the trial functions – is called the *Bubnov–Galerkin* method and is standard for self-adjoint problems such as linear elasticity.

3.9.2 Galerkin Weak Form for a Single Element

Substituting Equation (3.32) into Equation (3.33) and expanding:

$$\int_0^L N_i \left(-EA \frac{d^2 u^h}{dx^2} - p \right) dx = 0.$$

Integration by parts on the second-derivative term (the same step as Equation (3.28)) gives

$$\int_0^L EA \frac{dN_i}{dx} \frac{du^h}{dx} dx - \left[N_i EA \frac{du^h}{dx} \right]_0^L = \int_0^L N_i p dx. \quad (3.34)$$

The boundary term vanishes at nodes where the displacement is prescribed (kinematic admissibility requires $N_i = 0$ there), and reduces to the applied concentrated force F at a free end via the natural boundary condition $EA \, du/dx = F$. The result is the **Galerkin weak form**:

$$\int_0^L EA \frac{dN_i}{dx} \frac{du^h}{dx} dx = \int_0^L N_i p dx + F_i^{\text{natural}}. \quad (3.35)$$

3.9.3 Element Stiffness Matrix and Load Vector

Writing both rows ($i = 1$ and $i = 2$) simultaneously in matrix form,

$$\underbrace{\left[\int_0^L EA \mathbf{B}^\top \mathbf{B} dx \right]}_{\mathbf{k}^e} \mathbf{u}^e = \underbrace{\int_0^L \mathbf{N}^\top p dx}_{\mathbf{f}^e},$$

where $\mathbf{B} = d\mathbf{N}/dx = \frac{1}{L}[-1, 1]$ from Equation (3.10). Evaluating the integrals:

$$\mathbf{k}^e = \int_0^L EA \mathbf{B}^\top \mathbf{B} dx = \frac{EA}{L} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}, \quad (3.36)$$

$$\mathbf{f}^e = \int_0^L \mathbf{N}^\top p_0 dx = p_0 \int_0^L \begin{bmatrix} 1 - x/L \\ x/L \end{bmatrix} dx = \frac{p_0 L}{2} \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad (3.37)$$

in exact agreement with Equation (3.7) and Equation (3.20).

Key Result

The Galerkin finite element formulation yields

$$\mathbf{k}^e = \int_0^L EA \mathbf{B}^\top \mathbf{B} dx = \frac{EA}{L} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}, \quad \mathbf{f}^e = \int_0^L \mathbf{N}^\top p dx = \frac{p_0 L}{2} \begin{bmatrix} 1 \\ 1 \end{bmatrix},$$

identical to Equation (3.7) and Equation (3.20).

3.10 Equivalence of Galerkin and Virtual Work Formulations

The Galerkin weak form Equation (3.35) and the PVW equation Equation (3.30) are the same integral statement. To see this, write them side by side:

$$\begin{aligned} \text{PVW:} \quad & \int_0^L EA \frac{du}{dx} \frac{d\delta u}{dx} dx = \int_0^L p \delta u dx + \sum_i F_i \delta u(x_i), \\ \text{Galerkin FE:} \quad & \int_0^L EA \frac{du^h}{dx} \frac{dw_i}{dx} dx = \int_0^L p w_i dx + \sum_i F_i w_i(x_i). \end{aligned} \quad (3.38)$$

Setting $w_i = \delta u$, the two expressions are identical: both require the integral to vanish for every kinematically admissible test field, and both produce the same discrete system $\mathbf{k}^e \mathbf{u}^e = \mathbf{f}^e$.

Remark 3.7. PVW speaks the language of mechanics: δu is an infinitesimal virtual displacement, and the equation expresses the balance of virtual internal and external work. Galerkin speaks the language of weighted residuals: $w_i = N_i$ is a test function chosen to make the residual orthogonal to the approximation space. They are the same mathematical object. In the finite element context, both methods mandate choosing test functions from the same polynomial space as the trial – which is why they produce identical element matrices.

3.11 Principle of Minimum Potential Energy

3.11.1 Total Potential Energy of the Axial Bar

Definition 3.1. The **strain energy** stored in a linearly elastic axially loaded bar is

$$U = \frac{1}{2} \int_0^L EA \left(\frac{du}{dx} \right)^2 dx. \quad (3.39)$$

The **total potential energy** is

$$\Pi[u] = U - W_{\text{ext}}, \quad W_{\text{ext}} = \int_0^L p(x) u(x) dx + \sum_i F_i u(x_i). \quad (3.40)$$

The strain energy $U \geq 0$ is always non-negative. The total potential energy Π is a functional of the displacement field $u(x)$: each admissible u gives a scalar value of Π , and the equilibrium field is singled out by a stationarity condition.

3.11.2 Stationarity Condition and the Weak Form

Key Result

Principle of Minimum Potential Energy (PMPE): Of all kinematically admissible displacement fields, the one that satisfies equilibrium makes the total potential energy stationary:

$$\delta \Pi = 0 \quad (3.41)$$

for every admissible variation δu . For a stable elastic equilibrium, Π is in fact a global minimum.

Taking the first variation of Equation (3.40),

$$\delta \Pi = \int_0^L EA \frac{du}{dx} \frac{d\delta u}{dx} dx - \int_0^L p \delta u dx - \sum_i F_i \delta u(x_i) = 0. \quad (3.42)$$

Recognizing the first integral as δW_{int} and the remaining terms as δW_{ext} , this is precisely the PVW statement Equation (3.25):

$$\delta \Pi = 0 \iff \delta W_{\text{int}} = \delta W_{\text{ext}}. \quad (3.43)$$

Remark 3.8. PMPE is a special case of PVW valid for *conservative* systems, where the material is linearly elastic and the external loads are independent of the deformation. For the axially loaded bar with fixed applied loads, the two principles are exactly equivalent. PVW is the more general statement: it extends to non-conservative loading (e.g. follower loads) for which no potential energy exists.

3.11.3 Element Stiffness Matrix and Load Vector from PMPE

Substitute the finite element trial $u = \mathbf{N} \mathbf{u}^e$, $du/dx = \mathbf{B} \mathbf{u}^e$ into the element contributions to Equation (3.40). The strain energy becomes

$$U^e = \frac{1}{2} \int_0^L EA \mathbf{u}^{e\top} \mathbf{B}^\top \mathbf{B} \mathbf{u}^e dx = \frac{1}{2} \mathbf{u}^{e\top} \underbrace{\left[\int_0^L EA \mathbf{B}^\top \mathbf{B} dx \right]}_{\mathbf{k}^e} \mathbf{u}^e = \frac{1}{2} \mathbf{u}^{e\top} \mathbf{k}^e \mathbf{u}^e,$$

and the work of external loads (uniform p_0):

$$W_{\text{ext}}^e = \int_0^L (\mathbf{N} \mathbf{u}^e) p_0 dx = \mathbf{u}^{e\top} \underbrace{\left[\int_0^L \mathbf{N}^\top p_0 dx \right]}_{\mathbf{f}^e} = \mathbf{u}^{e\top} \mathbf{f}^e.$$

The element potential energy is therefore a quadratic function of the nodal displacements:

$$\Pi^e = \frac{1}{2} \mathbf{u}^{e\top} \mathbf{k}^e \mathbf{u}^e - \mathbf{u}^{e\top} \mathbf{f}^e. \quad (3.44)$$

Differentiating with respect to $\mathbf{u}^e$ and setting the result to zero,

$$\frac{\partial \Pi^e}{\partial \mathbf{u}^e} = \mathbf{k}^e \mathbf{u}^e - \mathbf{f}^e = \mathbf{0} \implies \mathbf{k}^e \mathbf{u}^e = \mathbf{f}^e. \quad (3.45)$$

The integrals evaluate to Equation (3.36) and Equation (3.37), so $\mathbf{k}^e$ and $\mathbf{f}^e$ are again identical to Equation (3.7) and Equation (3.20).

Key Result

Minimizing the element potential energy yields

$$\mathbf{k}^e = \int_0^L EA \mathbf{B}^\top \mathbf{B} dx = \frac{EA}{L} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}.$$

The stiffness matrix is the Hessian of the strain energy with respect to the nodal displacements.

3.11.4 Summary: Four Equivalent Paths to the Same Element Matrices

Four independent derivations all produce the same element stiffness matrix $\mathbf{k}^e$ (Equation (3.7)) and load vector $\mathbf{f}^e$ (Equation (3.20)).

The choice of method is a matter of convenience and generalizability. Direct equilibrium is intuitive for one-dimensional elements. PVW and Galerkin FE extend unchanged to two- and three-dimensional elements (Chapter 6); PMPE extends to any conservative system. Numerically, all four paths lead to the same linear system $\mathbf{K} \mathbf{u} = \mathbf{f}$, as confirmed by the MATLAB scripts Section A.3 and Section A.2.

Table 3.1: Four derivation paths for the bar element matrices.

Method	Starting point	Path to $\mathbf{k}^e$	Load vector $\mathbf{f}^e$
Direct Equilibrium	Hooke's law + nodal force balance	$N = EA\varepsilon$; collect into matrix form	Enriched trial + end reactions (Section 3.2)
PVW	$\delta W_{\text{int}} = \delta W_{\text{ext}}$ (Equation (3.25))	Substitute $u = \mathbf{N}\mathbf{u}^e$, $\delta u = \mathbf{N}\delta\mathbf{u}^e$	RHS of weak form (Section 3.8)
Galerkin FE	$\int N_i R \, dx = 0$, $w_i = N_i$ (Equation (3.33))	Integration by parts + $\mathbf{B}$ -matrix (Section 3.9)	RHS of weak form (Equation (3.37))
PMPE	$\delta\Pi = 0$, $\Pi = U - W_{\text{ext}}$ (Equation (3.40))	Substitute FE trial into U^e ; stationarity $\partial\Pi^e/\partial\mathbf{u}^e = \mathbf{0}$	$\partial W_{\text{ext}}^e/\partial\mathbf{u}^e$ (Section 3.11.3)

3.12 Natural Coordinates and Isoparametric Mapping

For generality, it is convenient to define a local (natural) coordinate $\xi \in [-1, 1]$ related to x by:

$$x = \frac{1-\xi}{2} x_1 + \frac{1+\xi}{2} x_2, \quad dx = \frac{L}{2} d\xi \quad (3.46)$$

The shape functions in natural coordinates become:

$$N_1(\xi) = \frac{1-\xi}{2}, \quad N_2(\xi) = \frac{1+\xi}{2} \quad (3.47)$$

This mapping is the foundation of the isoparametric concept used in Chapter 6.

Chapter 4

2D Truss Element

4.1 2D Truss Element

A truss element can carry axial loads only. In a 2D structure, each node has two translational DOFs: u (horizontal) and v (vertical). The element is inclined at angle θ with respect to the global x -axis [3].

4.1.1 Local Stiffness Matrix

In the local coordinate system aligned with the element axis, the stiffness matrix is identical to the 1D bar element Equation (3.7), expanded to account for the transverse DOF (which has zero stiffness):

$$\bar{\mathbf{k}}^e = \frac{EA}{L} \begin{bmatrix} 1 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 \\ -1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \quad (4.1)$$

where the DOF ordering is $[\bar{u}_1, \bar{v}_1, \bar{u}_2, \bar{v}_2]$.

4.1.2 Coordinate Transformation

The transformation matrix $\mathbf{T}$ rotates from global (x, y) to local $(\bar{x}, \bar{y})$ coordinates. With $c = \cos \theta$, $s = \sin \theta$:

$$\mathbf{T} = \begin{bmatrix} c & s & 0 & 0 \\ -s & c & 0 & 0 \\ 0 & 0 & c & s \\ 0 & 0 & -s & c \end{bmatrix} \quad (4.2)$$

4.1.3 Global Element Stiffness

The element stiffness in the global frame is:

$$\mathbf{k}^e = \mathbf{T}^T \bar{\mathbf{k}}^e \mathbf{T} = \frac{EA}{L} \begin{bmatrix} c^2 & cs & -c^2 & -cs \\ cs & s^2 & -cs & -s^2 \\ -c^2 & -cs & c^2 & cs \\ -cs & -s^2 & cs & s^2 \end{bmatrix} \quad (4.3)$$

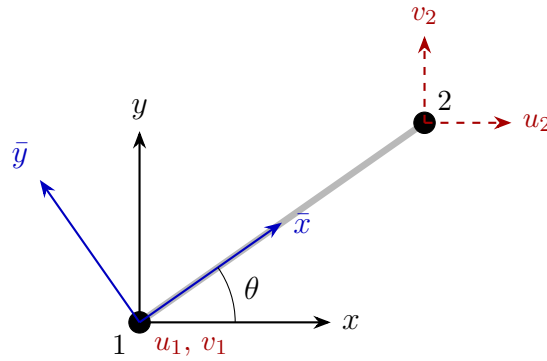


Figure 4.1: 2D truss element inclined at angle θ : global axes (x, y) and local axes $(\bar{x}, \bar{y})$.

Key Result

Angle convention: θ is measured counter-clockwise from the positive x -axis to the element axis (node 1 $\rightarrow$ node 2).

4.2 Worked Example: Three-Element Planar Truss

Example 4.1 (Three-element planar truss under vertical load). A planar truss consists of three elements and three nodes, as shown in Figure 4.2. All members have cross-sectional area $A = 225 \text{ mm}^2$ and Young's modulus $E = 210,000 \text{ N/mm}^2$. A vertical load $F = 10 \text{ kN}$ acts downward at the free node 2.

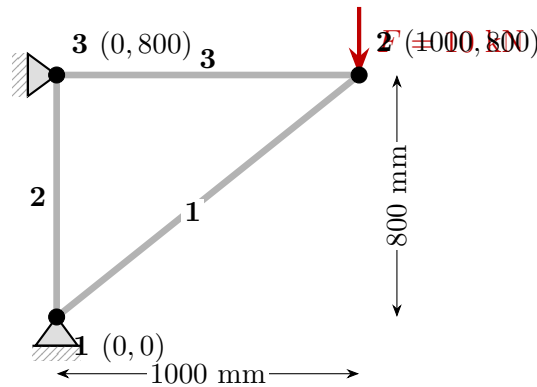


Figure 4.2: Three-node, three-element planar truss. Node coordinates in mm. Pin supports at nodes 1 and 3; 10 kN downward load at node 2.

Step 1: Element properties.

The DOF ordering is $[U_1, V_1, U_2, V_2, U_3, V_3] = \text{DOFs } [1, 2, 3, 4, 5, 6]$. Table 4.1 lists the geometric and stiffness properties for each element.

Step 2: Element global stiffness matrices.

Using Equation (4.3) with the values from Table 4.1:

Table 4.1: Element properties for the three-element truss.

Elem	Nodes	L (mm)	θ (°)	c	s	EA/L (N/mm)	DOFs
1	$1 \rightarrow 2$	1280.6	38.66	0.7809	0.6247	36 896	1,2,3,4
2	$1 \rightarrow 3$	800.0	90.00	0	1	59 063	1,2,5,6
3	$3 \rightarrow 2$	1000.0	0.00	1	0	47 250	5,6,3,4

$$\mathbf{k}^{(1)} = \begin{bmatrix} 22\,498 & 17\,998 & -22\,498 & -17\,998 \\ 17\,998 & 14\,398 & -17\,998 & -14\,398 \\ -22\,498 & -17\,998 & 22\,498 & 17\,998 \\ -17\,998 & -14\,398 & 17\,998 & 14\,398 \end{bmatrix} \text{ N/mm, DOFs [1, 2, 3, 4]}$$

$$\mathbf{k}^{(2)} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 59\,063 & 0 & -59\,063 \\ 0 & 0 & 0 & 0 \\ 0 & -59\,063 & 0 & 59\,063 \end{bmatrix} \text{ N/mm, DOFs [1, 2, 5, 6]}$$

$$\mathbf{k}^{(3)} = \begin{bmatrix} 47\,250 & 0 & -47\,250 & 0 \\ 0 & 0 & 0 & 0 \\ -47\,250 & 0 & 47\,250 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \text{ N/mm, DOFs [5, 6, 3, 4]}$$

Step 3: Global assembly.

Assembling the three element matrices into the 6×6 global stiffness:

$$\mathbf{K} = \begin{bmatrix} 22\,498 & 17\,998 & -22\,498 & -17\,998 & 0 & 0 \\ 17\,998 & 73\,461 & -17\,998 & -14\,398 & 0 & -59\,063 \\ -22\,498 & -17\,998 & 69\,748 & 17\,998 & -47\,250 & 0 \\ -17\,998 & -14\,398 & 17\,998 & 14\,398 & 0 & 0 \\ 0 & 0 & -47\,250 & 0 & 47\,250 & 0 \\ 0 & -59\,063 & 0 & 0 & 0 & 59\,063 \end{bmatrix} \text{ N/mm}$$

Note how $K_{22} = 14\,398 + 59\,063 = 73\,461$ receives contributions from both elements 1 and 2, and $K_{33} = 22\,498 + 47\,250 = 69\,748$ from elements 1 and 3.

Step 4: Boundary conditions and reduced system.

Nodes 1 and 3 are both pin supports: $U_1 = V_1 = U_3 = V_3 = 0$ (DOFs 1, 2, 5, 6 fixed). The only free DOFs are U_2 and V_2 (DOFs 3 and 4). The load vector has $F_{y2} = -10\,000$ N at DOF 4 and zero elsewhere. The reduced 2×2 system is:

$$\begin{bmatrix} 69\,748 & 17\,998 \\ 17\,998 & 14\,398 \end{bmatrix} \begin{bmatrix} U_2 \\ V_2 \end{bmatrix} = \begin{bmatrix} 0 \\ -10\,000 \end{bmatrix} \text{ N.} \quad (4.4)$$

Step 5: Solution.

Solving Equation (4.4) (see Section A.4 for the MATLAB implementation):

$$\begin{aligned} U_2 &= +0.2646 \text{ mm} \quad (\text{rightward}), \\ V_2 &= -1.0252 \text{ mm} \quad (\text{downward}). \end{aligned}$$

Step 6: Axial forces.

The axial force in each element is $N^e = (EA/L) \delta^e$ where $\delta^e = [-c, -s, c, s] \mathbf{u}^e$ is the axial elongation:

- **Element 1** (diagonal): $\delta_1 = 0.7809 \times 0.2646 + 0.6247 \times (-1.0252) = -0.4338$ mm,

$$N_1 = 36,896 \times (-0.4338) = -\mathbf{16,008} \text{ N (compression).}$$

- **Element 2** (vertical, between two fixed pins): $\delta_2 = s_2(V_3 - V_1) = 1 \times (0 - 0) = 0$,

$$N_2 = 0.$$

Both endpoints are fixed, so the element undergoes no axial deformation and carries no force under this loading.

- **Element 3** (horizontal): $\delta_3 = U_2 - U_3 = 0.2646$ mm,

$$N_3 = 47,250 \times 0.2646 = +\mathbf{12,500} \text{ N (tension).}$$

Step 7: Support reactions.

Multiplying the full 6×6 system $\mathbf{K} \mathbf{u} = \mathbf{f}_{\text{ext}}$ and extracting the fixed-DOF rows:

$$\begin{aligned} R_{1x} &= +12,500 \text{ N}, & R_{1y} &= +10,000 \text{ N}, \\ R_{3x} &= -12,500 \text{ N}, & R_{3y} &= 0 \text{ N}. \end{aligned}$$

Equilibrium check:

$$\sum F_x = 12,500 + (-12,500) = 0 \checkmark, \quad \sum F_y = 10,000 + 0 - 10,000 = 0 \checkmark.$$

Chapter 5

Euler-Bernoulli Beam and Frame Elements

5.1 Euler-Bernoulli Beam Element

The Euler-Bernoulli beam element models bending with the assumption that plane cross-sections remain plane and perpendicular to the neutral axis (no shear deformation).

5.1.1 Degrees of Freedom

Each node has two DOFs: transverse displacement v and rotation θ ($= dv/dx$). The element has 4 DOFs total: $\mathbf{u}^e = [v_1, \theta_1, v_2, \theta_2]^\top$.

5.1.2 Hermite Shape Functions

Cubic Hermite polynomials ensure C^1 continuity (continuous displacement and slope across elements):

$$\begin{aligned} H_1(x) &= 1 - 3\xi^2 + 2\xi^3 \\ H_2(x) &= L(\xi - 2\xi^2 + \xi^3) \\ H_3(x) &= 3\xi^2 - 2\xi^3 \\ H_4(x) &= L(-\xi^2 + \xi^3) \end{aligned} \tag{5.1}$$

where $\xi = x/L \in [0, 1]$.

5.1.3 Beam Stiffness Matrix

The curvature is $\kappa = d^2v/dx^2 = \mathbf{B}_b \mathbf{u}^e$ and the bending moment $M = EI\kappa$. The element stiffness is:

$$\mathbf{k}_b^e = \int_0^L EI \mathbf{B}_b^\top \mathbf{B}_b dx = \frac{EI}{L^3} \begin{bmatrix} 12 & 6L & -12 & 6L \\ 6L & 4L^2 & -6L & 2L^2 \\ -12 & -6L & 12 & -6L \\ 6L & 2L^2 & -6L & 4L^2 \end{bmatrix} \tag{5.2}$$

Key Result

$$\mathbf{k}_b^e = \frac{EI}{L^3} \begin{bmatrix} 12 & 6L & -12 & 6L \\ 6L & 4L^2 & -6L & 2L^2 \\ -12 & -6L & 12 & -6L \\ 6L & 2L^2 & -6L & 4L^2 \end{bmatrix}$$

5.1.4 Consistent Load Vector (Uniform Load q)

$$\mathbf{f}^e = \frac{qL}{12} \begin{bmatrix} 6 \\ L \\ 6 \\ -L \end{bmatrix} \quad (5.3)$$

5.2 Frame Element

A frame (beam-column) element combines axial bar stiffness and bending beam stiffness. In the local frame the DOFs are: $\mathbf{u}^e = [\bar{u}_1, \bar{v}_1, \bar{\theta}_1, \bar{u}_2, \bar{v}_2, \bar{\theta}_2]^\top$ and the local stiffness is the 6×6 block-diagonal combination of Equations (3.7) and (5.2).

The 6×6 transformation matrix:

$$\mathbf{T}_6 = \begin{bmatrix} c & s & 0 & 0 & 0 & 0 \\ -s & c & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & c & s & 0 \\ 0 & 0 & 0 & -s & c & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \quad (5.4)$$

The global frame element stiffness is $\mathbf{k}^e = \mathbf{T}_6^\top \bar{\mathbf{k}}_{\text{frame}}^e \mathbf{T}_6$.

5.3 Worked Example: Simply Supported Beam

Example 5.1 (Two-element simply supported beam). A beam with $L = 2$ m, $EI = 10^4$ N·m², simply supported at both ends, with central point load $P = 1000$ N.

Discretize with 2 equal beam elements ($L_e = 1$ m).

DOFs: node 1: (v_1, θ_1) , node 2: (v_2, θ_2) , node 3: (v_3, θ_3) .

BCs: $v_1 = 0, v_3 = 0$ (pin supports).

Free DOFs: $[\theta_1, v_2, \theta_2, \theta_3]$.

Analytical mid-span: $v_2 = PL^3/(48EI) = 8.33 \times 10^{-4}$ m.

See Section A.5 in the Appendix for the MATLAB implementation.

Chapter 6

Two-Dimensional Continuum Elements

6.1 Plane Elasticity

Two-dimensional continuum elements model thin plates or long prismatic bodies [1, 4]. Two assumptions are common:

Plane stress Thin plates loaded in their plane: $\sigma_z = \tau_{xz} = \tau_{yz} = 0$.

Plane strain Long bodies with constrained out-of-plane strain: $\varepsilon_z = \gamma_{xz} = \gamma_{yz} = 0$.

6.1.1 Stress and Strain Vectors

For both cases, the in-plane state is described by:

$$\boldsymbol{\sigma} = \begin{bmatrix} \sigma_x \\ \sigma_y \\ \tau_{xy} \end{bmatrix}, \quad \boldsymbol{\varepsilon} = \begin{bmatrix} \varepsilon_x \\ \varepsilon_y \\ \gamma_{xy} \end{bmatrix} \quad (6.1)$$

6.1.2 Constitutive Matrix

Plane stress (ν = Poisson's ratio, E = Young's modulus):

$$\mathbf{D}_{\text{ps}} = \frac{E}{1 - \nu^2} \begin{bmatrix} 1 & \nu & 0 \\ \nu & 1 & 0 \\ 0 & 0 & \frac{1-\nu}{2} \end{bmatrix} \quad (6.2)$$

Plane strain:

$$\mathbf{D}_{\text{pe}} = \frac{E}{(1 + \nu)(1 - 2\nu)} \begin{bmatrix} 1 - \nu & \nu & 0 \\ \nu & 1 - \nu & 0 \\ 0 & 0 & \frac{1-2\nu}{2} \end{bmatrix} \quad (6.3)$$

6.2 Constant Strain Triangle (CST)

The CST (or T3) is the simplest 2D element: 3 nodes, 2 DOFs per node (6 total), with linear displacement fields and constant strains throughout.

6.2.1 Shape Functions

For a triangle with nodes (x_1, y_1) , (x_2, y_2) , (x_3, y_3) and element area A_e , the shape functions are the *area coordinates*:

$$N_i(x, y) = \frac{1}{2A_e}(a_i + b_i x + c_i y), \quad i = 1, 2, 3 \quad (6.4)$$

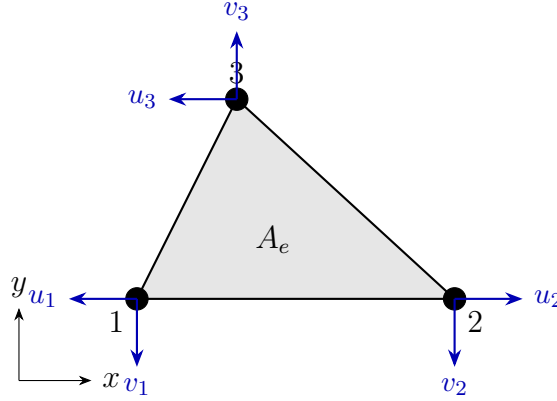


Figure 6.1: CST element: 3 nodes, 2 DOFs per node (u_i, v_i) , element area A_e .

where:

$$a_1 = x_2 y_3 - x_3 y_2, \quad b_1 = y_2 - y_3, \quad c_1 = x_3 - x_2$$

$$2A_e = \det \begin{bmatrix} 1 & x_1 & y_1 \\ 1 & x_2 & y_2 \\ 1 & x_3 & y_3 \end{bmatrix}$$

(and cyclic permutations for $i = 2, 3$).

6.2.2 Strain-Displacement Matrix

$$\boldsymbol{\varepsilon} = \mathbf{B} \mathbf{u}^e, \quad \mathbf{B} = \frac{1}{2A_e} \begin{bmatrix} b_1 & 0 & b_2 & 0 & b_3 & 0 \\ 0 & c_1 & 0 & c_2 & 0 & c_3 \\ c_1 & b_1 & c_2 & b_2 & c_3 & b_3 \end{bmatrix} \quad (6.5)$$

Note that $\mathbf{B}$ is constant $\Rightarrow$ strain is constant over the CST element.

6.2.3 Element Stiffness Matrix

$$\mathbf{k}^e = \int_{A_e} \mathbf{B}^\top \mathbf{D} \mathbf{B} t \, dA = t A_e \mathbf{B}^\top \mathbf{D} \mathbf{B} \quad (6.6)$$

where t is the element thickness.

Key Result

The CST stiffness is evaluated in *closed form* (no numerical integration required) because $\mathbf{B}$ and $\mathbf{D}$ are both constant over the element:

$$\mathbf{k}^e = t A_e \mathbf{B}^\top \mathbf{D} \mathbf{B} \in \mathbb{R}^{6 \times 6}$$

6.3 Isoparametric Quadrilateral Element (Q4)

The 4-node quadrilateral (Q4) uses the same polynomials to interpolate both geometry and displacement — this is the *isoparametric* concept.

6.3.1 Shape Functions in Natural Coordinates

In the reference square $(\xi, \eta) \in [-1, 1]^2$:

$$N_i(\xi, \eta) = \frac{1}{4}(1 + \xi_i \xi)(1 + \eta_i \eta), \quad i = 1, 2, 3, 4 \quad (6.7)$$

where (ξ_i, η_i) are the corner coordinates: $(-1, -1), (1, -1), (1, 1), (-1, 1)$.

6.3.2 Jacobian Mapping

The physical coordinates are mapped as:

$$x = \sum_{i=1}^4 N_i x_i, \quad y = \sum_{i=1}^4 N_i y_i \quad (6.8)$$

The Jacobian matrix $\mathbf{J}$ relates derivatives:

$$\mathbf{J} = \begin{bmatrix} \frac{\partial x}{\partial \xi} & \frac{\partial y}{\partial \xi} \\ \frac{\partial x}{\partial \eta} & \frac{\partial y}{\partial \eta} \end{bmatrix} = \sum_{i=1}^4 \begin{bmatrix} \frac{\partial N_i}{\partial \xi} \\ \frac{\partial N_i}{\partial \eta} \end{bmatrix} \begin{bmatrix} x_i & y_i \end{bmatrix} \quad (6.9)$$

Physical derivatives are recovered as:

$$\begin{bmatrix} \frac{\partial N_i}{\partial x} \\ \frac{\partial N_i}{\partial y} \end{bmatrix} = \mathbf{J}^{-1} \begin{bmatrix} \frac{\partial N_i}{\partial \xi} \\ \frac{\partial N_i}{\partial \eta} \end{bmatrix} \quad (6.10)$$

6.3.3 Numerical Integration (Gauss Quadrature)

The element stiffness integral is evaluated by 2×2 Gauss quadrature (sufficient for Q4):

$$\mathbf{k}^e = t \int_{-1}^1 \int_{-1}^1 \mathbf{B}(\xi, \eta)^\top \mathbf{D} \mathbf{B}(\xi, \eta) |\mathbf{J}| d\xi d\eta \approx t \sum_{i=1}^{n_g} \sum_{j=1}^{n_g} w_i w_j \mathbf{B}(\xi_i, \eta_j)^\top \mathbf{D} \mathbf{B}(\xi_i, \eta_j) |\mathbf{J}(\xi_i, \eta_j)| \quad (6.11)$$

For 2×2 Gauss: $n_g = 2$, $\xi_i = \pm 1/\sqrt{3}$, $w_i = 1$.

Order	Points per direction	Exact for polynomials up to
1	1	degree 1
2	2	degree 3
3	3	degree 5

6.4 Worked Example: CST Plate in Tension

Example 6.1 (Single CST element under tension). A single CST element: nodes at $(0, 0)$, $(1, 0)$, $(0, 1)$, thickness $t = 0.01$ m, $E = 200 \times 10^9$ Pa, $\nu = 0.3$, plane stress. Apply $\sigma_x = 100$ MPa (force $F = 0.5$ MN at nodes 2 and 3).

$A_e = 0.5$ m², $\mathbf{B}$: constant from Equation (6.5), $\mathbf{D}$: from Equation (6.2). See Section A.6 in the Appendix for the MATLAB implementation.

Chapter 7

Structural Dynamics and Vibration

7.1 Equations of Motion

When inertia and damping effects are included, the FEM system of equations becomes:

$$\mathbf{M} \ddot{\mathbf{u}} + \mathbf{C} \dot{\mathbf{u}} + \mathbf{K} \mathbf{u} = \mathbf{f}(t) \quad (7.1)$$

where $\mathbf{M}$ is the global *mass matrix*, $\mathbf{C}$ is the *damping matrix*, $\mathbf{K}$ is the *stiffness matrix*, and $\mathbf{f}(t)$ is the time-varying external force.

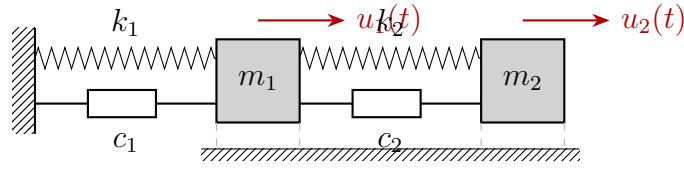


Figure 7.1: Two-DOF mass-spring-damper system corresponding to Equation (7.1) with matrices $\mathbf{M}$, $\mathbf{C}$, $\mathbf{K}$.

7.2 Mass Matrix

7.2.1 Consistent Mass Matrix

Derived from the same shape functions used for the stiffness matrix. For a bar element:

$$\mathbf{m}^e = \int_0^L \rho A \mathbf{N}^T \mathbf{N} dx = \frac{\rho AL}{6} \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \quad (7.2)$$

For the Euler-Bernoulli beam element with 4 DOFs $[v_1, \theta_1, v_2, \theta_2]$:

$$\mathbf{m}_b^e = \frac{\rho AL}{420} \begin{bmatrix} 156 & 22L & 54 & -13L \\ 22L & 4L^2 & 13L & -3L^2 \\ 54 & 13L & 156 & -22L \\ -13L & -3L^2 & -22L & 4L^2 \end{bmatrix} \quad (7.3)$$

7.2.2 Lumped Mass Matrix

A diagonal approximation obtained by *row-summing* the consistent mass matrix and placing the total on the diagonal:

$$\mathbf{m}_{\text{lump}}^e = \frac{\rho AL}{2} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad (7.4)$$

The lumped mass matrix is computationally efficient but less accurate for bending-dominated problems.

Remark 7.1. Rotational DOF lumped masses are typically set to zero (or a small fraction), which can cause rank-deficiency issues. The consistent mass matrix is generally preferred for structural dynamics.

7.3 Free Vibration: Eigenvalue Problem

For free undamped vibration ($\mathbf{C} = 0$, $\mathbf{f} = 0$), assuming harmonic response $\mathbf{u}(t) = \boldsymbol{\phi} e^{i\omega t}$:

$$\mathbf{K} \boldsymbol{\phi} = \omega^2 \mathbf{M} \boldsymbol{\phi} \iff (\mathbf{K} - \omega^2 \mathbf{M}) \boldsymbol{\phi} = \mathbf{0} \quad (7.5)$$

Key Result

Generalized eigenvalue problem: $\mathbf{K} \boldsymbol{\Phi} = \mathbf{M} \boldsymbol{\Phi} \boldsymbol{\Omega}^2$

In MATLAB: `[Phi, Lambda] = eig(K, M)` where `diag(Lambda) = \omega_n^2`.

The n natural frequencies $\omega_1 \leq \omega_2 \leq \dots \leq \omega_n$ and corresponding mode shapes $\boldsymbol{\phi}_i$ satisfy:

$$\boldsymbol{\Phi}^T \mathbf{M} \boldsymbol{\Phi} = \mathbf{I}, \quad \boldsymbol{\Phi}^T \mathbf{K} \boldsymbol{\Phi} = \text{diag}(\omega_i^2) \quad (7.6)$$

(mass-normalized modes).

7.4 Modal Analysis

7.4.1 Modal Decomposition

Expand the response in modal coordinates $\mathbf{q}(t)$:

$$\mathbf{u}(t) = \boldsymbol{\Phi} \mathbf{q}(t) \quad (7.7)$$

Substituting into Equation (7.1) and using orthogonality Equation (7.6), each mode decouples into a SDOF equation:

$$\ddot{q}_i + 2\zeta_i \omega_i \dot{q}_i + \omega_i^2 q_i = f_i^*(t) \quad (7.8)$$

where $f_i^*(t) = \boldsymbol{\phi}_i^T \mathbf{f}(t)$ is the modal force and ζ_i is the modal damping ratio.

7.4.2 Damping: Rayleigh Model

Proportional (Rayleigh) damping assigns damping as a linear combination of mass and stiffness:

$$\mathbf{C} = \alpha \mathbf{M} + \beta \mathbf{K} \quad (7.9)$$

The coefficients α and β are determined from two target damping ratios ζ_1 , ζ_2 at frequencies ω_1 , ω_2 :

$$\begin{bmatrix} \zeta_1 \\ \zeta_2 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 1/\omega_1 & \omega_1 \\ 1/\omega_2 & \omega_2 \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix} \quad (7.10)$$

7.5 Time Integration: Newmark- β Method

For direct time integration of Equation (7.1), the Newmark- β scheme [6, 2] uses:

$$\mathbf{u}_{n+1} = \mathbf{u}_n + \Delta t \dot{\mathbf{u}}_n + \Delta t^2 \left[\left(\frac{1}{2} - \beta \right) \ddot{\mathbf{u}}_n + \beta \ddot{\mathbf{u}}_{n+1} \right] \quad (7.11)$$

$$\dot{\mathbf{u}}_{n+1} = \dot{\mathbf{u}}_n + \Delta t [(1 - \gamma) \ddot{\mathbf{u}}_n + \gamma \ddot{\mathbf{u}}_{n+1}] \quad (7.12)$$

Parameters	Scheme	Stability	Accuracy
$\beta = 0, \gamma = 1/2$	Central difference	Conditional	2nd order
$\beta = 1/4, \gamma = 1/2$	Average acceleration	Unconditional	2nd order
$\beta = 1/6, \gamma = 1/2$	Linear acceleration	Conditional	2nd order

7.5.1 Effective Stiffness Matrix

At each time step solve the effective system:

$$\hat{\mathbf{K}} \mathbf{u}_{n+1} = \hat{\mathbf{f}}_{n+1} \quad (7.13)$$

where:

$$\hat{\mathbf{K}} = \mathbf{K} + \frac{\gamma}{\beta \Delta t} \mathbf{C} + \frac{1}{\beta \Delta t^2} \mathbf{M} \quad (7.14)$$

$$\begin{aligned} \hat{\mathbf{f}}_{n+1} = & \mathbf{f}_{n+1} + \mathbf{M} \left(\frac{1}{\beta \Delta t^2} \mathbf{u}_n + \frac{1}{\beta \Delta t} \dot{\mathbf{u}}_n + \left(\frac{1}{2\beta} - 1 \right) \ddot{\mathbf{u}}_n \right) \\ & + \mathbf{C} \left(\frac{\gamma}{\beta \Delta t} \mathbf{u}_n + \left(\frac{\gamma}{\beta} - 1 \right) \dot{\mathbf{u}}_n + \frac{\Delta t}{2} \left(\frac{\gamma}{\beta} - 2 \right) \ddot{\mathbf{u}}_n \right) \end{aligned} \quad (7.15)$$

7.6 Connection to 2-DOF System

The 2-DOF mass-spring-damper system analyzed in the companion MATLAB scripts (`spectrum_response_2DOF.m`, `harmonic_response_2DOF.m`) is a direct application of Equation (7.1) with:

$$\mathbf{M} = \begin{bmatrix} m_1 & 0 \\ 0 & m_2 \end{bmatrix}, \quad \mathbf{C} = \begin{bmatrix} c_1 + c_2 & -c_2 \\ -c_2 & c_2 \end{bmatrix}, \quad \mathbf{K} = \begin{bmatrix} k_1 + k_2 & -k_2 \\ -k_2 & k_2 \end{bmatrix}$$

The natural frequencies computed by `eig(K,M)` yield $f_{n1} = 2.031$ Hz and $f_{n2} = 4.321$ Hz — exactly the resonance peaks observed in the frequency response plots.

See Section A.7 in the Appendix for a Newmark- β MATLAB implementation applied to the 2-DOF system.

Appendix A

MATLAB Code Listings

This appendix provides MATLAB implementations for the key algorithms presented in each chapter. All scripts are self-contained and use only core MATLAB (no additional toolboxes required).

A.1 1D Bar Element Assembly and Solution

```
1 % fem_bar_1d.m -- 3-element 1D bar, tip load
2 % Example from Chapter 2
3
4 clear; clc;
5
6 %% Parameters
7 nElem = 3;           % number of elements
8 nNode = nElem + 1;   % number of nodes
9 L      = 0.9;         % total length [m]
10 EA     = 70e6;        % axial stiffness [N]
11 F_tip  = 10000;       % tip force [N]
12 Le     = L / nElem;   % element length [m]
13
14 %% Element stiffness matrix
15 ke = (EA/Le) * [1 -1; -1 1];
16
17 %% Global assembly
18 K = zeros(nNode, nNode);
19 for e = 1:nElem
20     dofs = [e, e+1]; % global DOFs for element e
21     K(dofs, dofs) = K(dofs, dofs) + ke;
22 end
23
24 %% Force vector
25 f = zeros(nNode, 1);
26 f(nNode) = F_tip; % tip load at last node
27
28 %% Apply BC: node 1 fixed (u1 = 0)
29 freeDofs = 2:nNode;
30 K_red = K(freeDofs, freeDofs);
31 f_red = f(freeDofs);
```

```

32
33 %% Solve
34 u_free = K_red \ f_red;
35 u = [0; u_free]; % full displacement vector
36
37 %% Post-processing
38 fprintf('Nodal displacements [m]:\n');
39 for i = 1:nNode
40     fprintf(' u(%d) = %.6e m\n', i, u(i));
41 end

```

Listing A.1: Three-element bar under tip load (Chapter 3)

A.2 Stepped Bar with Distributed and Concentrated Loads

This listing reproduces the worked example of Example 3.2: a two-element cantilever bar with a step in cross-section, a distributed axial load on the first element, and concentrated forces at nodes 2 and 3. The script assembles the global stiffness and force vectors element by element, applies the fixed boundary condition, solves the reduced system, recovers the reaction, and plots the displacement and internal-force fields using the enriched trial Equation (3.12).

```

1 % stepped_bar_2elem.m -- two-element stepped bar (Chapter 3)
2 % Cantilever bar with two segments:
3 %   E1: L1 = 500 mm, area A, distributed load p = -4 N/mm
4 %   E2: L2 = 400 mm, area 2A, no distributed load
5 % Concentrated forces: +6000 N at node 2, -15000 N at node 3.
6
7 clear; clc; close all;
8
9 %% Material and geometry
10 E = 210000; % Young's modulus [MPa = N/mm^2]
11 A = 225; % element-1 cross-section area [mm^2]
12 p_e1 = -4; % distributed load on element 1 [N/mm]
13 p_e2 = 0; % distributed load on element 2 [N/mm]
14
15 %% Concentrated forces
16 P_node2 = 6000; % +x at node 2 [N]
17 P_node3 = -15000; % -x at node 3 [N]
18
19 %% Discretization
20 nodes_x = [0, 500, 900]; % node coordinates [mm]
21 nElem = 2;
22 nNode = numel(nodes_x);
23 nDof = nNode; % 1D bar: 1 DOF per node
24 elem_conn = [1 2; 2 3]; % element connectivity
25 elem_area = [A, 2*A];
26 elem_p = [p_e1, p_e2];
27 elem_L = diff(nodes_x);

```



```

28
29 %% Initialize globals
30 KG      = zeros(nDof, nDof);           % global stiffness
31 FG_p    = zeros(nDof, 1);             % distributed-load contribution
32 FG_c    = zeros(nDof, 1);             % concentrated forces
33
34 %% Element-by-element assembly
35 for e = 1:nElem
36     L      = elem_L(e);
37     Ae     = elem_area(e);
38     pe     = elem_p(e);
39     dofs   = elem_conn(e,:);
40     ke     = (E*Ae/L) * [1 -1; -1 1];
41     fe_p   = (pe*L/2) * [1; 1];
42     KG(dofs, dofs) = KG(dofs, dofs) + ke;
43     FG_p(dofs)      = FG_p(dofs)      + fe_p;
44 end
45
46 %% Add concentrated forces and form total
47 FG_c(2) = P_node2;
48 FG_c(3) = P_node3;
49 FG = FG_p + FG_c;
50
51 %% Boundary conditions: node 1 fixed
52 fixed_dofs = 1;
53 free_dofs  = setdiff(1:nDof, fixed_dofs);
54
55 UG          = zeros(nDof, 1);
56 UG(free_dofs) = KG(free_dofs, free_dofs) \ FG(free_dofs);
57 RG          = KG*UG - FG;
58
59 fprintf('UG = '); disp(UG');
60 fprintf('RG = '); disp(RG');
61
62 %% Element-level postprocessing using the enriched trial:
63 %    $u(x) = N1*u1 + N2*u2 + p*x*(L-x)/(2EA)$ 
64 %    $N(x) = EA*(u2-u1)/L + p*(L-2x)/2$ 
65 xg = []; ug = []; Ng = [];
66 for e = 1:nElem
67     L      = elem_L(e);
68     Ae     = elem_area(e);
69     pe     = elem_p(e);
70     dofs   = elem_conn(e,:);
71     u1     = UG(dofs(1)); u2 = UG(dofs(2));
72     x      = linspace(0, L, 200);
73     xi     = x / L;
74     u_e    = (1-xi)*u1 + xi*u2 + pe*x.*(L - x)/(2*E*Ae);
75     N_e    = E*Ae*(u2-u1)/L + pe*(L - 2*x)/2;
76     xg     = [xg, nodes_x(dofs(1)) + x];
77     ug     = [ug, u_e];
78     Ng     = [Ng, N_e];

```

```

79 end
80
81 %% Plots
82 figure('Color','w'); plot(xg, ug, 'LineWidth', 1.5); grid on;
83 xlabel('x [mm]'); ylabel('u(x) [mm]');
84 title('Displacement along the stepped bar');
85
86 figure('Color','w'); plot(xg, Ng, 'LineWidth', 1.5); grid on;
87 xlabel('x [mm]'); ylabel('N(x) [N]');
88 title('Internal axial force along the stepped bar');

```

Listing A.2: Stepped bar with distributed and concentrated loads (Chapter 3)

A.3 Verification of the Principle of Virtual Work

This listing supports Example 3.3: it takes the solved stepped bar (the same u_1, u_2, u_3) and, for three different kinematically admissible virtual displacements, evaluates the internal and external virtual work separately. All three cases satisfy $\delta W_{\text{int}} = \delta W_{\text{ext}}$ to numerical precision, confirming the principle of virtual work.

```

1  % pvw_verification_stepped_bar.m
2  % Numerically verify the principle of virtual work for the
   SOLVED
3  % stepped bar (the worked example of Section "Stepped Bar").
4  % For three different kinematically admissible virtual
   displacements,
5  % the script computes the internal and external virtual work
   separately
6  % and shows they are equal.
7
8  clear; clc;
9
10 %% Stepped bar parameters
11 E = 210000;
12 A = 225;
13 EA1 = E*A;          % element 1 axial stiffness [N]
14 EA2 = E*2*A;        % element 2 axial stiffness [N]
15 L1 = 500;           % element 1 length [mm]
16 L2 = 400;           % element 2 length [mm]
17 p1 = -4;            % element 1 distributed load [N/mm]
18 F2 = 6000;          % concentrated force at node 2 [N]
19 F3 = -15000;         % concentrated force at node 3 [N]
20
21 %% Exact (FEM) nodal displacements from the Stepped Bar example
22 u1 = 0;              % fixed
23 u2 = -20/189;        % mm    (= -0.10582)
24 u3 = -32/189;        % mm    (= -0.16931)
25
26 %% Internal axial force in each element using the enriched
   trial:
27 %   N(x) = EA*(u_b - u_a)/L + p*(L - 2x_local)/2

```

```

28 N1 = @(x) EA1*(u2-u1)/L1 + p1*(L1 - 2*x)/2;           % x measured
    from node 1
29 N2 = @(x) EA2*(u3-u2)/L2 + 0*x;                       % x measured
    from node 2
30
31 %% Three admissible virtual displacements (each must satisfy du
    (x=0)=0)
32 % Each entry stores du(x), du'(x), and the nodal values du(
    node_k)
33 testCases = struct( ...
34     'name',      {}, ...
35     'du',        {}, ...
36     'du_dx',     {}, ...
37     'du_n2',     {}, ...
38     'du_n3',     {});
39
40 % Case A -- tent: du_2 = 1, du_3 = 0
41 testCases(end+1) = struct( ...
42     'name',      'Case A: (du_1, du_2, du_3) = (0, 1, 0)', ...
43     'du',        @(x_local, elem) (elem==1).*(x_local/L1) + (elem==2)
        .*(1 - x_local/L2), ...
44     'du_dx',     @(x_local, elem) (elem==1)*(1/L1)          + (elem
        ==2)*(-1/L2), ...
45     'du_n2',     1, ...
46     'du_n3',     0);
47
48 % Case B -- ramp + plateau: du_2 = 1, du_3 = 1
49 testCases(end+1) = struct( ...
50     'name',      'Case B: (du_1, du_2, du_3) = (0, 1, 1)', ...
51     'du',        @(x_local, elem) (elem==1).*(x_local/L1) + (elem==2)
        .*1, ...
52     'du_dx',     @(x_local, elem) (elem==1)*(1/L1)          + (elem
        ==2)*0, ...
53     'du_n2',     1, ...
54     'du_n3',     1);
55
56 % Case C -- global quadratic du(x_glob) = (x_glob / Ltot)^2
57 Ltot = L1 + L2;
58 testCases(end+1) = struct( ...
59     'name',      'Case C: du(x) = (x / 900)^2 (smooth, non-
        piecewise-linear)', ...
60     'du',        @(x_local, elem) ((elem==1).*x_local + (elem==2).*(
        L1 + x_local)).^2 / Ltot^2, ...
61     'du_dx',     @(x_local, elem) 2*((elem==1).*x_local + (elem==2)
        .*(L1 + x_local)) / Ltot^2, ...
62     'du_n2',     (L1/Ltot)^2, ...
63     'du_n3',     1);
64
65 %% Verify PVW for each case
66 fprintf('Principle of Virtual Work verification -- stepped bar\
    n');

```

```

67 fprintf('Solved nodal displacements:  u1 = 0,  u2 = %.5f,  u3 =
        %.5f mm\n\n', u2, u3);
68 fprintf('%-58s %14s %14s %14s\n', 'Admissible virtual
        displacement', ...
69         'dW_int [Nmm]', 'dW_ext [Nmm]', '|diff|');
70 fprintf('%s\n', repmat('-', 1, 102));
71
72 for k = 1:length(testCases)
73     tc = testCases(k);
74
75     % Internal virtual work -- sum element contributions
76     dWint_e1 = integral(@(x) N1(x).*tc.du_dx(x,1), 0, L1);
77     dWint_e2 = integral(@(x) N2(x).*tc.du_dx(x,2), 0, L2);
78     dWint     = dWint_e1 + dWint_e2;
79
80     % External virtual work -- distributed load (element 1 only
81     % and
82     % concentrated forces; reaction at node 1 contributes 0
83     % because du(0)=0.
84     dWext_d   = integral(@(x) p1.*tc.du(x,1), 0, L1);
85     dWext_c   = F2*tc.du_n2 + F3*tc.du_n3;
86     dWext     = dWext_d + dWext_c;
87
88     fprintf('%-58s %14.5f %14.5f %14.2e\n', tc.name, dWint,
89         dWext, abs(dWint-dWext));
90 end
91
92 fprintf(['\nFor every kinematically admissible virtual
93     displacement,\n', ...
94         'dW_int = dW_ext to numerical precision: PVW holds.\n'
95     ]);

```

Listing A.3: PVW verification for the stepped bar (Chapter 3)

A.4 2D Truss Analysis

```

1  % fem_truss_2d.m -- Chapter 4: Worked Example, 3-element 2D
   % truss
2  % Nodes: 1=(0,0), 2=(1000,800), 3=(0,800) [mm]
3  % BCs:   node 1 pinned (U1=V1=0), node 3 pinned (U3=V3=0)
4  % Load: Fy = -10000 N (downward) at node 2
5  clear; clc; close all;
6
7  coords = [ 0      0;
8            1000  800;
9            0    800];
10 conn = [1 2; 1 3; 3 2];
11 E = 210000; A = 225;
12 nEl = 3; nDOF = 6;
13

```

```

14 for e = 1:nEl
15     n1 = conn(e,1); n2 = conn(e,2);
16     dx = coords(n2,1) - coords(n1,1);
17     dy = coords(n2,2) - coords(n1,2);
18     Le(e) = sqrt(dx^2 + dy^2);
19     ce(e) = dx / Le(e);
20     se(e) = dy / Le(e);
21     EAL(e) = E * A / Le(e);
22 end
23
24 K = zeros(nDOF);
25 for e = 1:nEl
26     c = ce(e); s = se(e); k = EAL(e);
27     ke = k * [ c^2    c*s   -c^2   -c*s;
28               c*s    s^2   -c*s   -s^2;
29               -c^2   -c*s    c^2    c*s;
30               -c*s   -s^2    c*s    s^2];
31     n1 = conn(e,1); n2 = conn(e,2);
32     dofs = [2*n1-1, 2*n1, 2*n2-1, 2*n2];
33     K(dofs, dofs) = K(dofs, dofs) + ke;
34 end
35
36 F = zeros(nDOF, 1);
37 F(4) = -10000;
38 fixedDOFs = [1 2 5 6];
39 freeDOFs = [3 4];
40
41 u_free = K(freeDOFs, freeDOFs) \ F(freeDOFs);
42 u = zeros(nDOF, 1);
43 u(freeDOFs) = u_free;
44
45 for e = 1:nEl
46     c = ce(e); s = se(e);
47     n1 = conn(e,1); n2 = conn(e,2);
48     dofs = [2*n1-1, 2*n1, 2*n2-1, 2*n2];
49     N(e) = EAL(e) * ([-c -s c s] * u(dofs));
50 end
51
52 R = K * u - F;
53
54 fprintf('U2 = %.4f mm, V2 = %.4f mm\n', u(3), u(4));
55 fprintf('N1 = %.1f N, N2 = %.1f N, N3 = %.1f N\n', N(1), N(2),
56         N(3));
57 fprintf('R1x = %.1f N, R1y = %.1f N\n', R(1), R(2));
58 fprintf('R3x = %.1f N, R3y = %.1f N\n', R(5), R(6));
59
60 scale = 500;
61 figure; hold on; axis equal;
62 for e = 1:nEl
63     n1 = conn(e,1); n2 = conn(e,2);
64     xo = [coords(n1,1) coords(n2,1)];

```

```

64     yo = [coords(n1,2) coords(n2,2)];
65     plot(xo, yo, 'k--', 'LineWidth', 1.2);
66     plot(xo + scale*[u(2*n1-1) u(2*n2-1)], ...
67          yo + scale*[u(2*n1)   u(2*n2)   ], 'b-', 'LineWidth',
          2.5);
68 end
69 xlabel('x [mm]'); ylabel('y [mm]');
70 title(sprintf('Deformed shape (x%d)', scale));
71 exportgraphics(gcf, '../figures/truss2d_deformed.pdf', '
    ContentType', 'vector');
72 exportgraphics(gcf, '../figures/truss2d_deformed.png', '
    Resolution', 300);

```

Listing A.4: Three-element planar truss (Chapter 4)

A.5 2D Beam Element

```

1  % fem_beam_2d.m -- 2-element Euler-Bernoulli beam
2  % Example from Chapter 3
3
4  clear; clc;
5
6  %% Parameters
7  nElem = 2;
8  nNode = nElem + 1;
9  L_total = 2.0;           % total beam length [m]
10 EI      = 1e4;           % bending stiffness [N*m^2]
11 P       = 1000;          % central point load [N]
12 Le      = L_total / nElem;
13
14 %% Beam element stiffness (4 DOF: v1, th1, v2, th2)
15 ke = (EI/Le^3) * [12,    6*Le,   -12,    6*Le;
16                  6*Le,  4*Le^2, -6*Le,  2*Le^2;
17                  -12,   -6*Le,   12,    -6*Le;
18                  6*Le,  2*Le^2, -6*Le,  4*Le^2];
19
20 %% DOFs: each node has [v, theta]
21 nDOF = 2 * nNode;
22 K = zeros(nDOF, nDOF);
23
24 for e = 1:nElem
25     n1 = e; n2 = e + 1;
26     dofs = [2*n1-1, 2*n1, 2*n2-1, 2*n2];
27     K(dofs, dofs) = K(dofs, dofs) + ke;
28 end
29
30 %% Force vector: central load at node 2 (v direction)
31 f = zeros(nDOF, 1);
32 f(2*(nElem/2+1)-1) = P;
33

```



```

27 ke = t * Ae * B' * D * B;
28
29 sigma_app = 100e6;
30 F_total = sigma_app * 1 * t;
31 f = zeros(6, 1);
32 f(3) = F_total / 2;
33 f(5) = F_total / 2;
34
35 freeDOFs = 3:6;
36 u_free = ke(freeDOFs, freeDOFs) \ f(freeDOFs);
37 u = zeros(6, 1);
38 u(freeDOFs) = u_free;
39
40 eps_el = B * u;
41 sig_el = D * eps_el;
42 fprintf('sigma_x = %.4e\nsigma_y = %.4e\ntau_xy = %.4e\n', ...
43         sig_el(1), sig_el(2), sig_el(3));

```

Listing A.6: Single CST element under plane stress (Chapter 6)

A.7 Newmark- β Time Integration for 2-DOF System

```

1 % fem_newmark_2dof.m -- Newmark-beta time integration
2 % 2-DOF mass-spring-damper system from Chapter 5
3
4 clear; clc;
5
6 m1 = 1500; m2 = 1000;
7 k1 = 600000; k2 = 300000;
8 c1 = 1000; c2 = 1000;
9
10 M = [m1, 0; 0, m2];
11 C = [c1+c2, -c2; -c2, c2];
12 K = [k1+k2, -k2; -k2, k2];
13
14 beta = 1/4;
15 gamma = 1/2;
16
17 f_exc = 2.031;
18 omega = 2*pi*f_exc;
19 t_end = 5;
20 dt = 1e-3;
21 t_vec = 0:dt:t_end;
22 nSteps = length(t_vec);
23
24 F_amp = [1000; 1200];
25 F = @(t) F_amp * sin(omega * t);
26

```



```

27 u      = zeros(2, 1);
28 v      = zeros(2, 1);
29 a      = M \ (F(0) - C*v - K*u);
30
31 K_eff = K + (gamma/(beta*dt))*C + (1/(beta*dt^2))*M;
32
33 U = zeros(2, nSteps);
34 U(:,1) = u;
35
36 for n = 1:nSteps-1
37     f_eff = F(t_vec(n+1)) ...
38         + M * (1/(beta*dt^2)*u + 1/(beta*dt)*v + (1/(2*beta)-1)
39             *a) ...
40         + C * (gamma/(beta*dt)*u + (gamma/beta-1)*v + dt/2*(
41             gamma/beta-2)*a);
42
43     u_new = K_eff \ f_eff;
44     a_new = 1/(beta*dt^2)*(u_new - u) - 1/(beta*dt)*v - (1/(2*
45         beta)-1)*a;
46     v_new = v + dt*((1-gamma)*a + gamma*a_new);
47
48     U(:, n+1) = u_new;
49     u = u_new; v = v_new; a = a_new;
50 end
51
52 figure('Name','Newmark Transient Response');
53 plot(t_vec, U(1,:)*1000, 'b'); hold on;
54 plot(t_vec, U(2,:)*1000, 'r');
55 xlabel('Time [s]'); ylabel('Displacement [mm]');
56 legend('u_1(t)','u_2(t)'); grid on;

```

Listing A.7: Newmark- β transient response of the 2-DOF system (Chapter 7)

Bibliography

- [1] Klaus-Jürgen Bathe. *Finite Element Procedures*. Prentice Hall, Englewood Cliffs, NJ, 2006.
- [2] Ray W. Clough and Joseph Penzien. *Dynamics of Structures*. McGraw-Hill, New York, 2nd edition, 1993.
- [3] Robert D. Cook, David S. Malkus, Michael E. Plesha, and Robert J. Witt. *Concepts and Applications of Finite Element Analysis*. John Wiley & Sons, New York, 4th edition, 2002.
- [4] Thomas J. R. Hughes. *The Finite Element Method: Linear Static and Dynamic Finite Element Analysis*. Dover Publications, Mineola, NY, 2000.
- [5] Daryl L. Logan. *A First Course in the Finite Element Method*. Cengage Learning, 5th edition, 2011.
- [6] Nathan M. Newmark. A method of computation for structural dynamics. *Journal of the Engineering Mechanics Division, ASCE*, 85(EM3):67–94, 1959.
- [7] O. C. Zienkiewicz, R. L. Taylor, and J. Z. Zhu. *The Finite Element Method: Its Basis and Fundamentals*. Elsevier Butterworth-Heinemann, Oxford, 6th edition, 2005.